

Multimotion Visual Odometry (MVO)

©The Author(s) 2021
Manuscript #IJR-21-4311

Kevin M. Judd¹ and Jonathan D. Gammell¹

Abstract

Visual motion estimation is a well-studied challenge in autonomous navigation. Recent work has focused on addressing multimotion estimation in highly dynamic environments. These environments not only comprise multiple, complex motions but also tend to exhibit significant occlusion.

Estimating third-party motions simultaneously with the sensor egomotion is difficult because an object's observed motion consists of both its true motion and the sensor motion. Most previous works in multimotion estimation simplify this problem by relying on appearance-based object detection or application-specific motion constraints. These approaches are effective in specific applications and environments but do not generalize well to the full multimotion estimation problem (MEP).

This paper presents Multimotion Visual Odometry (MVO), a multimotion estimation pipeline that estimates the full $SE(3)$ trajectory of *every* motion in the scene, including the sensor egomotion, without relying on appearance-based information. MVO extends the traditional visual odometry (VO) pipeline with multimotion segmentation and tracking techniques. It uses physically founded motion priors to extrapolate motions through temporary occlusions and identify the reappearance of motions through *motion closure*. Evaluations on real-world data from the Oxford Multimotion Dataset (OMD) and the KITTI Vision Benchmark Suite demonstrate that MVO achieves good estimation accuracy compared to similar approaches and is applicable to a variety of multimotion estimation challenges.

Keywords

Motion Estimation, Motion Segmentation, Visual Odometry, Structure from Motion, Multiple Object Tracking

1 Introduction

Autonomous robotic platforms are being deployed in an increasingly diverse range of applications. Many platforms and algorithms are highly specialized to operate in specific environments, but the ability to safely navigate diverse and complex dynamic environments is becoming more important.

A principle objective of robotic navigation systems is determining the egomotion of a dynamic robot relative to its environment, usually using a mounted sensor. Visual odometry (VO) is a technique for finding the egomotion of a camera by isolating the static parts of a scene and estimating the motion relative to those regions (Moravec 1980). The segmentation of static portions of the scene is itself an important focus of visual navigation research (e.g., Nistér 2004; Fraundorfer and Scaramuzza 2012), but much less research has focused on also analyzing the dynamic regions of the scene that the segmentation rejects.

Third-party dynamic motions are more difficult to estimate than the sensor egomotion because their observed motions comprise both their arbitrary real-world motions and the sensor egomotion. This multimotion estimation problem (MEP) is often simplified by constraining motions according to kinematic assumptions (e.g., Sabzevari and Scaramuzza 2016), or by first isolating and estimating the sensor egomotion and then compensating for it to estimate the remaining third-party motions in the scene (e.g., Jaimez et al. 2017). These techniques can be successful in specific applications, but few generalized approaches have been proposed to address the full MEP.

Third-party motions are also difficult to track because highly dynamic scenes often include significant occlusions. Occlusions can be defined as any time there are insufficient measurements of part of a scene. They are either direct, when an object is partially or fully obscured, or indirect, due to sensor limitations or algorithmic failure. Multiobject tracking (MOT) techniques can successfully track multiple dynamic objects through full and partial occlusions, but they are often constrained by application-specific object detectors and simplistic motion models (e.g., Milan et al. 2016; Mitzel et al. 2010). Consistently estimating multiple independent motions in the presence of occlusions is necessary for autonomous navigation in complex, dynamic environments.

This paper presents Multimotion Visual Odometry (MVO), a multimotion estimation pipeline that prioritizes motion over appearance. It emphasizes the importance of understanding *how* things are moving in the environment before understanding *what* they are. MVO addresses the MEP by applying multilabeling techniques to the traditional VO pipeline using only a rigid-motion assumption. It

¹Estimation, Search, and Planning Group, Oxford Robotics Institute, University of Oxford, UK

Corresponding author:

Kevin Judd, Estimation, Search, and Planning Group Oxford Robotics Institute, University of Oxford, Oxford OX2 6NN, UK.

<https://robotic-esp.com/kjudd>

Email: kjudd@oxfordrobotics.institute



Figure 1. Motion trajectories estimated by MVO for sequences from the KITTI (top, Geiger et al. 2012) and OMD (bottom, Judd and Gammell 2019a) datasets. The KITTI segment involves an autonomous driving scenario in a residential environment where the car-mounted camera follows a van and a cyclist. The OMD segment includes four independently swinging blocks observed by a handheld camera. In both sequences, the $SE(3)$ trajectories of the camera egomotion and third-party motions are estimated simultaneously without prior knowledge of their number, appearance, or nature.

simultaneously estimates the full $SE(3)$ trajectory of every motion in a scene, including the sensor egomotion, without making any *a priori* assumptions about object number, appearance, or motion (Fig. 1). The pipeline is adaptable to a variety of motion priors and achieves robust motion-based tracking through occlusion via *motion closure*.

The rest of the paper is organized as follows. Section 2 explores the details of the MEP and the current approaches for addressing it. Section 3 describes how the MVO pipeline extends traditional VO techniques to multimotion estimation. Section 4 demonstrates how different types of state estimators and motion priors can be used within the MVO pipeline. Section 5 discusses how to address the challenges involved in implementing MVO as a sliding-window estimator, particularly with respect to occlusions. Sections 6 and 7 evaluate and discuss the performance of the MVO pipeline quantitatively and qualitatively using the Oxford Multimotion Dataset (OMD; Judd and Gammell 2019a) and the KITTI Vision Benchmark Suite (Geiger et al. 2012).

This paper is a continuation of ideas that were first presented at the Joint Industry and Robotics CDTs Symposium (Judd et al. 2018b) and the Long-term Human Motion Prediction Workshop (Judd and Gammell 2019b) and were published in Judd et al. (2018a), Judd (2019), and Judd and Gammell (2020). It makes the following specific contributions:

- Presents a unified and updated version of MVO that is adaptable to a variety of trajectory representations and estimation techniques.
- Incorporates a continuous $SE(3)$ white-noise-on-jerk (WNOJ) prior into the estimator, extends it for geocentric third-party estimation, and examines its

advantages and limitations in the context of the MEP, along with the previously presented discrete and white-noise-on-acceleration (WNOA) models.

- Details the challenges of full-batch and sliding-window implementations of MVO, including handling temporary occlusions.
- Compares pose-only, pose-velocity, and pose-velocity-acceleration estimators both quantitatively on indoor experiments (OMD) and qualitatively in the real world (KITTI).

2 Background

Dynamic environments consist of the static background, a moving observer (i.e., sensor), and one or more independent, third-party motions. The observed motions include both the objects' real-world motion and the sensor egomotion, so the static scene appears to move with the inverse of the egomotion trajectory.

Fully addressing this MEP requires both *segmentation*, i.e., clustering points according to their movement between observations, and *estimation*, i.e., calculating the motion of a cluster of points. The interdependence of these tasks creates a *chicken-and-egg* problem: the motion trajectories must be estimated from groups of points, but the segmentation of those groups depends on the available motion estimates.

This interdependence is addressed in single-motion VO systems by using heuristics (e.g., number of features) to select the egomotion and ignore the other motions in the scene (Section 2.1). Appearance-based tracking techniques are often used to track multiple dynamic objects, but currently do not accurately estimate the underlying motion of generic objects (Section 2.2). These heuristic- and appearance-based techniques are not readily extensible to general *multimotion* estimation problems and analyzing multiple independently moving bodies remains a challenging problem for state-of-the-art vision systems (Section 2.3).

2.1 Egomotion Estimation

VO estimates the motion of a camera relative to its static environment (i.e., *egomotion*) and has a long history in robotics starting with the motion-estimation pipeline presented by Moravec (1980). The camera egomotion is calculated from a stream of images by assuming it corresponds to the dominant motion in the scene.

The egomotion is estimated by first isolating regions of the scene that move according to the dominant motion and assuming they are static. Those regions are then used to estimate the egomotion trajectory relative to those static points and the rest of the scene is usually ignored as noise.

Recent developments in VO focus on improving the static segmentation (Nistér 2004; Sabzevari and Scaramuzza 2016). The estimation can also be improved by using more advanced estimation techniques, such as dense or direct estimation methods (Valgaerts et al. 2012; Engel et al. 2017) and continuous $SE(3)$ motion priors (Anderson and Barfoot 2015; Tang et al. 2019). These techniques make the egomotion estimation more robust and accurate, but few methods extend them to also analyze the dynamic parts of the scene that VO rejects.

2.2 Multiobject Tracking (MOT)

MOT techniques are often used in tandem with VO to track multiple dynamic objects over time, usually through a *tracking-by-detection* paradigm (Milan et al. 2016). They use appearance-based object detectors to find instances of specific object classes in each frame. They then seek to accurately associate present and past detections to form simple $\mathbb{R}^3$ trajectories. These techniques are limited by the quality of their detectors, and are often constrained to tracking application-specific object classes.

MOT algorithms often track objects through occlusions by modeling object interactions (Yang et al. 2011) or employing simple motion models to extrapolate object positions (Mitzel et al. 2010). These techniques are limited by the object representations used by the target detectors, and tend to be constrained to simple motion models that do not adequately represent the motion of an object. Fully addressing the MEP requires applying more expressive motion estimation techniques, such as those used to estimate egomotion, to the other third-party motions in a scene.

2.3 Multimotion Estimation

In contrast to MOT, multimotion estimation focuses on the motions in the scene rather than the objects that generate them. This involves both segmenting observed points into independent rigid objects and estimating the $SE(3)$ trajectories of those objects. Several multimotion estimation approaches have been developed using techniques such as scene flow clustering, simultaneous localization and mapping (SLAM) frameworks, and energy-minimization, but none has fully addressed the general MEP.

Lenz et al. (2011) use sparse scene flow to detect and track multiple dynamic objects from a dynamic, vehicle-mounted camera. The approach operates on stereo image pairs and clusters sparse points based on their scene flow. These clusters are then used to track objects through $\mathbb{R}^3$ space. The approach is not limited to tracking specific classes of objects and is robust to some partial occlusions, but it requires objects to be of limited size and in contact with the ground plane. Scene flow also estimates pixel-wise translational motion, so estimating the motion of rotating objects requires further segmentation and estimation.

Menze and Geiger (2015) instead model the world as piecewise-planar *superpixels*. The approach simultaneously calculates both scene flow and the homographies defining the motion of the planes in the scene, which are modeled as full $SE(3)$ motions. Similarly, Quiroga et al. (2014) and Jaimez et al. (2017) define optimization frameworks that estimate scene flow by modeling the underlying motions as full $SE(3)$ motions. This formulation is more robust to rotations but it complicates the MEP by introducing additional constraints to estimate the pixel-wise velocity of every point in the scene. Each of these techniques also relies on accurate RGB-D sensing and an independent egomotion estimation pipeline, which can lead to errors when large portions of the scene are moving.

Wang et al. (2007) extend the traditional SLAM formulation to include MOT techniques in their SLAMMOT framework using lidar sensors and monocular and stereo cameras (Lin and Wang 2010). The traditional SLAM state, consisting of the $SE(3)$ sensor pose and a static map of the environment,

is extended to include the range, bearing, and linear velocity of any tracked objects in the scene. Like most 3D MOT algorithms, SLAMMOT only estimates the $\mathbb{R}^3$ position of third-party objects, rather than their full $SE(3)$ pose. CubeSLAM (Yang and Scherer 2019) similarly extends the SLAM framework to track the full $SE(3)$ pose of 3D bounding boxes for each detected object, but not the pose of the object itself.

Rather than tracking 3D bounding boxes, Huang et al. (2019) estimate bulk point motions in ClusterSLAM, a multibody dynamic SLAM backend that was extended to a full multimotion estimation pipeline in ClusterVO (Huang et al. 2020). The pipeline clusters tracked 3D points according to their bulk motion but relies on a semantic object detector to inform its data association and tracking. Visual Direct Object-aware SLAM (VDO-SLAM; Henein et al. 2020; Zhang et al. 2020a,b) also extends the SLAM formulation to estimate the full $SE(3)$ pose of other objects in the scene, but uses an external instance segmentation algorithm to segment the image into its constituent objects first. DynaSLAM II (Bescos et al. 2021) similarly uses front-end instance segmentation to guide feature matching and segmentation before estimating the $SE(3)$ trajectory of each tracked object. ClusterVO, VDO-SLAM, and DynaSLAM II are able to estimate the motion of each object more accurately than techniques that only track bounding boxes, but they are reliant on significant front-end preprocessing, which limits their applicability to the general MEP. In contrast, DymSLAM (Wang et al. 2020) is a SLAM framework without any significant front-end processing. It uses spectral clustering to segment dynamic motions and estimate their $SE(3)$ motions but primarily focuses on dense scene reconstruction which is separate from the MEP.

Instead of building on the traditional SLAM framework, Isack and Boykov (2012) demonstrate an energy-based multimodel-fitting framework that can be used to segment motions in a frame-to-frame manner. Roussos et al. (2012) use this framework to simultaneously estimate depth maps and object motions from a monocular camera while also performing object tracking. The technique minimizes a complex cost function involving photometric consistency, geometric smoothness in depth, spatial smoothness in image space, and a minimum-description-length term that promotes a compact solution. The approach proceeds in an offline, full-batch manner and its additional focus on dense reconstruction, combined with its initialization requirements, make it ill-suited for MEP applications.

Other techniques use RGB-D sensors to segment and estimate motions while also performing suitable tracking for object reconstruction. Rünz and Agapito (2017) use an RGB-D camera to segment and track targets while simultaneously fusing 3D object models. The technique combines motion segmentation with object instance segmentation, which relies on predefined class-based object detectors. Rünz et al. (2018) extend this work to real-time processing by improving the efficiency of the semantic segmentation, and Xu et al. (2019) define a similar system using a volumetric representation, rather than surface normals. These techniques represent significant progress in addressing the MEP, with the added ability to fuse 3D models of the tracked objects; but they are reliant on high-quality, dense depth sensing and they are limited in the number of active models they can reasonably process.

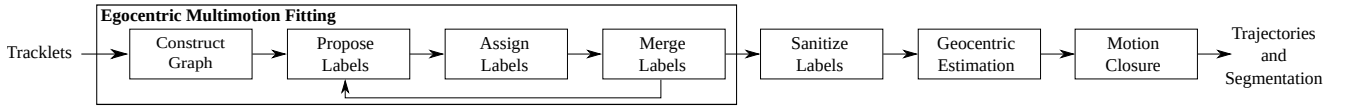


Figure 2. An illustration of the stereo MVO pipeline, which extends the standard VO pipeline by replacing the egomotion estimator with a multimotion estimator. MVO operates on 3D tracklets and generates the $SE(3)$ trajectory for *every* motion in the scene, including the sensor egomotion. The pipeline builds a neighborhood graph based on how rigidly pairs of points move over time and iteratively splits and estimates new labels using the graph. It assigns labels based on an energy functional, and merges labels that can be considered redundant until convergence. Once the segmentation converges, the labels are sanitized and a batch estimation produces the geocentric $SE(3)$ trajectories, employing *motion closure* to determine if newly discovered motions can be explained by the reappearance of an occluded object.

Qiu et al. (2019) and Eckenhoff et al. (2019) address the MEP using monocular cameras. Monocular observations are underconstrained and a separate scale parameter must be estimated using the inertial measurement unit (IMU). This scale parameter is valid for the egomotion of the camera, but estimating the scale for third-party motions is difficult and leads to several degenerate cases, such as when the object and camera motions are colinear. The performance of these techniques is impressive given the limitations of the sensors, but they are not broadly applicable to the general MEP.

In contrast to these approaches, MVO fully addresses the MEP in an online manner using only a rigid-motion assumption. The pipeline estimates the full $SE(3)$ trajectory of every motion in a complex, dynamic scene without any *a priori* information about object number, appearance, or motion. It also operates directly on sparse 3D points and is therefore applicable to a variety of 3D sensors. MVO can incorporate physically founded $SE(3)$ motion priors to extrapolate motions accurately through occlusions and achieves motion-based tracking through a motion closure procedure.

3 Simultaneous $SE(3)$ Estimation and Segmentation

Fully addressing the MEP requires both motion estimation and segmentation. Motion estimation involves calculating the motion of a set of points. Motion segmentation involves clustering points according to their movement.

The interdependence of estimation and segmentation leads to a *chicken-and-egg* problem: the motion trajectories must be estimated from groups of points, but the segmentation of those groups depends on the available motion estimates. In single-motion VO systems this is addressed by using heuristics (e.g., number of features) to select the egomotion and ignore the other motions in the scene. These heuristics are not readily extensible to *multimotion* estimation problems and analyzing multiple independently moving bodies remains a challenging problem for state-of-the-art vision systems.

MVO addresses the MEP by casting motion segmentation as a multilabeling problem where a label represents a motion trajectory. It iteratively segments and estimates motions using these labels in an alternating fashion until the segmentation converges. The set of motion labels adapts to the motions in the scene without making any *a priori* assumptions about object number, appearance, or motion.

MVO does not use any assumptions or heuristics to identify the static background until after the segmentation converges, so each motion is estimated as if it corresponds to the

true egomotion. After the segmentation converges, the true egomotion is selected and used to reestimate the true third-party motions in a geocentric frame (Section 4).

The pipeline operates directly on tracked 3D observations, i.e., *tracklets*, from a 3D sensor. It iteratively generates labels and segments those observations according to their motion (Fig. 2). The tracklets are embedded in a graph structure that forms the basis of the motion segmentation (Section 3.2). New labels are proposed when the motion of tracklets assigned to a given label may be more accurately explained by multiple trajectories (Section 3.3). Motion labels are assigned to each tracklet by minimizing a cost functional that incorporates reprojection residual, graph smoothness, and model complexity (Section 3.4). Tracklets whose motions are not well explained by any other label are assigned to an outlier label and redundant and oversegmented labels are then merged (Section 3.4.4). Once the segmentation converges, the final labels are then sanitized and any remaining outliers are rejected (Section 3.5). Initial versions of these ideas were first presented in Judd et al. (2018a) and Judd et al. (2018b), and they are further developed here.

3.1 Notation

A sequence of observations of a point, j , relative to a moving sensor frame, $\mathcal{F}_C$, over multiple time steps forms a *tracklet*

$$\mathcal{P}^j := \left(\mathbf{p}_{C_k}^{j C_k} \right)_{k=1 \dots K} = \left(\begin{bmatrix} \mathbf{p}_{C_k}^{j C_k} \\ 1 \end{bmatrix} \right)_{k=1 \dots K},$$

where C_k refers to the observing sensor frame at time k (Fig. 3, right), and $\mathbf{p}_{C_k}^{j C_k} \in \mathbb{R}^{4 \times 1}$ is the homogeneous representation of the Euclidean vector, $\mathbf{p}_{C_k}^{j C_k} \in \mathbb{R}^{3 \times 1}$.

A transform, $\mathbf{T}_{AB} \in SE(3)$, relates the current coordinate frame, $\mathcal{F}_A$, to the origin frame $\mathcal{F}_B$. The transform is defined blockwise,

$$\mathbf{T}_{AB} := \begin{bmatrix} \mathbf{C}_{AB} & \mathbf{p}_A^{BA} \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (1)$$

where $\mathbf{C}_{AB}$ is the rotation matrix to $\mathcal{F}_A$ from $\mathcal{F}_B$, and $\mathbf{p}_A^{BA}$ is the translation to $\mathcal{F}_B$ from $\mathcal{F}_A$ expressed in $\mathcal{F}_A$. This translation can also be expressed as

$$\mathbf{p}_A^{BA} = -\mathbf{C}_{AB} \mathbf{p}_B^{AB}. \quad (2)$$

The motion trajectory, $\mathcal{T}_A$, consists of a sequence of $SE(3)$ transforms,

$$\mathcal{T}_A := \left(\mathbf{T}_{A_k A_1} \right)_{k=2 \dots K},$$

that relate the pose at time k to a privileged initial pose $\mathcal{F}_{A_1}$ (Fig. 3, left). The trajectory can also be modeled continuously as $\mathbf{T}_A(t)$ such that $\mathbf{T}_{A_k A_1} := \mathbf{T}_A(t_k)$. The frames, $\mathcal{F}_A$ and

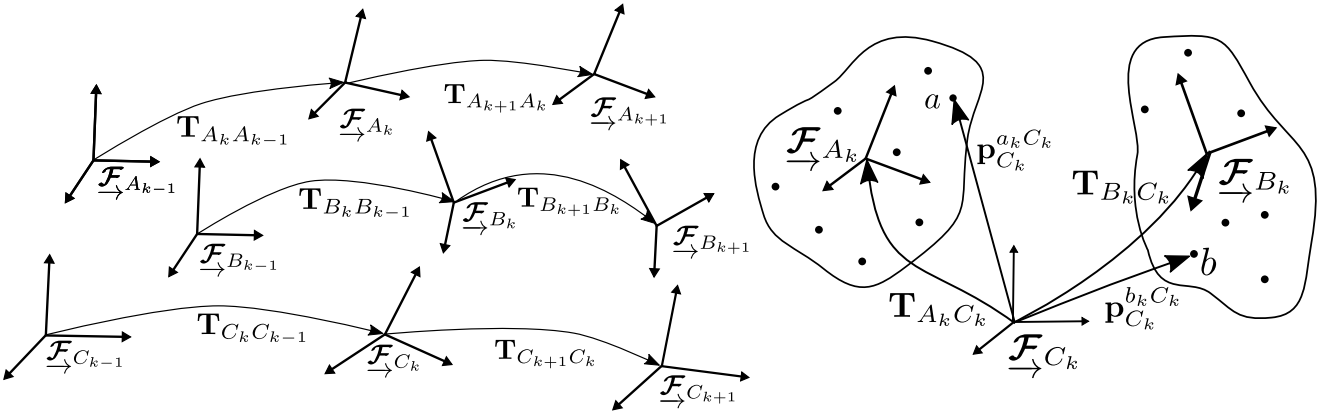


Figure 3. Illustrations of the MEP showing the motion of frames through time (left) and the relative point observations (right). Two independent third-party motions, $\mathcal{F}_A$ and $\mathcal{F}_B$, are observed by a moving camera, $\mathcal{F}_C$, through feature measurements on the objects, $\{\mathbf{p}_{C_k}^{a_k C_k}\}$ and $\{\mathbf{p}_{C_k}^{b_k C_k}\}$. Solving the MEP requires simultaneously segmenting and estimating the motions of these measurements.

$\mathcal{F}_B$, can be the sensor frame, $\mathcal{F}_C$, or an arbitrary third-party motion frame, $\mathcal{F}_\ell$.

The set of tracklets visible in a scene, $\mathcal{P}$, comprises the subsets of tracklets assigned to each motion label, $\mathcal{P}_\ell$, i.e., the support for label ℓ . The current set of all motion labels available for assignment is given by $\mathcal{L}$.

3.2 Graph Construction

The multilabeling is performed on a graph that represents each tracklet as a vertex. The graph, $\mathcal{N}$, embodies the rigid-body assumption by defining the cost, ω_{ij} , between pairs of vertices as the variance in the $\mathbb{R}^3$ distance between their associated tracklets over time,

$$\omega_{ij} := \frac{1}{K} \sum_{k=1}^K \left(\left\| \mathbf{p}_{C_k}^{i_k C_k} - \mathbf{p}_{C_k}^{j_k C_k} \right\| - \bar{d}_{ij} \right)^2, \quad (3)$$

where $\bar{d}_{ij}$ is the mean distance between $\mathbf{p}^i$ and $\mathbf{p}^j$. The variance between tracklets moving similarly is small, making this edge cost small for tracklets on the same rigid body.

The graph consists of vertices for all tracklets and the k least-costly edges from each vertex. This k -nearest neighbors graph is unlikely to connect independently moving pairs of tracklets so it reasonably and efficiently approximates the rigid-body assumption. Its connectivity forms the basis for label generation (Section 3.3) and assignment (Section 3.4), and an over- or under-connected graph can lead to an under- or overfit solution, respectively.

3.3 Label Proposal

The motions in the scene are segmented by assigning tracklets to a finite set of motion labels, $\mathcal{L}$. This set must be updated regularly because dynamic environments are constantly changing. New labels are proposed by splitting existing labels whenever their motions could be explained by multiple trajectories.

A potential new label, ℓ' , is generated for each fully-disjoint component of the subgraph, $\mathcal{N}_\ell \subseteq \mathcal{N}$, defined by the current label support, $\mathcal{P}_\ell$. This prioritizes graph smoothness while allowing large label subgraphs consisting of distinct rigid motions to be split into new labels.

3.3.1 Estimating New Labels Each label is estimated as if it corresponds to the static portion of the scene using standard egomotion techniques. The new egomotion hypothesis, $\ell' \mathcal{T}_C$, is calculated from the dominant motion of the corresponding subgraph component using frame-to-frame RANSAC, which segments the inliers and outliers of that motion (Fischler and Bolles 1981).

The frame-to-frame RANSAC procedure estimates the observed motion between pairs of consecutive frames, $\ell' \mathbf{T}_{C_k C_{k-1}}$, by sampling three tracklets in both frames and assuming they are static. The transform is estimated in a blockwise manner according to (1).

The translational component is calculated by

$$\ell' \mathbf{p}_{C_k}^{C_{k-1} C_k} = -\ell' \mathbf{p}_{C_k}^{C_k C_{k-1}} = -\left(\bar{\mathbf{p}}_{C_k} - \ell' \mathbf{C}_{C_k C_{k-1}} \bar{\mathbf{p}}_{C_{k-1}} \right),$$

where $\bar{\mathbf{p}}_{C_k}$ is the centroid of the sampled points at time k . The rotation is found by solving Wahba's problem (Wahba 1965) using singular value decomposition (Markley 1988),

$$\mathbf{U} \Sigma \mathbf{V}^T := \sum_{j=1}^3 \left(\mathbf{p}_{C_{k-1}}^{j_{k-1} C_{k-1}} - \bar{\mathbf{p}}_{C_{k-1}} \right) \left(\mathbf{p}_{C_k}^{j_k C_k} - \bar{\mathbf{p}}_{C_k} \right)^T, \\ \ell' \mathbf{C}_{C_k C_{k-1}} = \mathbf{U} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \|\mathbf{U}\| \|\mathbf{V}\| \end{bmatrix} \mathbf{V}^T.$$

The calculated transform is evaluated by the number of inlier tracklets with reprojection residuals,

$$e_k \left(\mathbf{p}^j, \ell' \mathbf{T}_{C_k C_{k-1}} \right) := \left\| \mathbf{s} \left(\mathbf{p}_{C_k}^{j_k C_k} \right) - \mathbf{s} \left(\ell' \mathbf{T}_{C_k C_{k-1}} \mathbf{p}_{C_{k-1}}^{j_{k-1} C_{k-1}} \right) \right\|, \quad (4)$$

less than a chosen threshold, e_{th} , where $\mathbf{s}(\cdot)$ applies the sensor projection (e.g., nonlinear stereo camera; Appendix B).

The process of sampling, estimation, and evaluation is iterated a chosen number of times, N_{RSAC} , and the transform with the most inliers is kept. This process is repeated independently for each pair of consecutive frames in the estimation window to generate a new trajectory for the label.

This frame-to-frame procedure is used to propose a new trajectory for each extant label. Any tracklets found to be outliers of the newly estimated trajectories are appended to the outlier label, $\mathcal{O}$. After the outliers are removed from all existing label sets, new labels are generated from the outlier label using the same proposal process.

3.4 Label Assignment

Casting the MEP as a multilabeling problem involves assigning each tracklet, $\mathcal{p} \in \mathcal{P}$, to a motion label, $\ell \in \mathcal{L}$, such that some energy functional, $E(\mathcal{L})$, is minimized. This functional incorporates both the fidelity of a label to the motion of a tracklet and a piecewise-rigid model of the environment. A complexity term is included to penalize using additional labels and incentivize a compact solution. The energy functional balances these residual error (Section 3.4.1), label smoothness (Section 3.4.2), and label complexity (Section 3.4.3) terms,

$$E(\mathcal{L}) := \underbrace{\sum_{\mathcal{p} \in \mathcal{P}} \rho(\mathcal{p}, \ell(\mathcal{p}))}_{\text{Residual}} + \underbrace{\lambda_{\text{sm}} \sum_{(\mathcal{p}^i, \mathcal{p}^j) \in \mathcal{N}} \exp(-\omega_{ij}) V(\mathcal{p}^i, \mathcal{p}^j)}_{\text{Smoothness}} + \underbrace{\sum_{\ell \in \mathcal{L}} \mu_{\ell} \psi(\ell)}_{\text{Complexity}} \quad (5)$$

where $\ell(\mathcal{p})$ gives the label currently assigned to $\mathcal{p}$ and λ_{sm} is a user-selected proportionality parameter. Increasing the proportionality parameter can lead to a spatially smoother solution at the cost of ignoring small, but clearly distinct, motions. Decreasing the parameter can isolate small motions but can also lead to more spurious motion detections.

The residual and smoothness portions of the energy are minimized using the convex relaxation algorithm (CORAL; Amayo et al. 2018), a multimodel-fitting algorithm that relaxes discrete labeling to allow for efficient parallelization on GPGPU devices. This “soft” labeling assigns each tracklet a score for how applicable each label is to the observed data. The score is in the continuous range $[0, 1]$ and the scores for each tracklet across all labels sum to 1. The soft labels are discretized by taking the strongest label for each point. Points with ambiguous soft labels (e.g., less than 0.5) are labeled as outliers to avoid mislabeling points. The total energy is finally minimized by merging similar labels to avoid oversegmentation and overfitting (Section 3.4.4).

3.4.1 Residual Cost

The residual term,

$$\sum_{\mathcal{p} \in \mathcal{P}} \rho(\mathcal{p}, \ell(\mathcal{p})),$$

penalizes labels that poorly describe their assigned data. It is defined as the sum of the residual errors in applying the label trajectories to tracklets. The residual for each point-label pair is defined as

$$\rho(\mathcal{p}, \ell) := \max_{k \in t_1 \dots t_2} e_k \left(\mathcal{p}, {}^{\ell} \mathbf{T}_{C_k C_j} \right),$$

where e_k is defined in (4) and t_1 and t_2 are the first and last time stamps that the tracklet is observed, respectively. Maximum error penalizes tracklets that do not fit a motion, even for a single frame. This conservative labeling helps preserve the accuracy of the trajectory estimation.

The residual cost of the outlier label, $\mathcal{O}$, is uniquely defined to have low cost for points not well explained by existing labels. The cost decays exponentially with the best-fitting label cost,

$$\rho(\mathcal{p}, \mathcal{O}) := \alpha \exp \left(-\frac{1}{\beta} \min_{\ell \in \mathcal{L}} \rho(\mathcal{p}, \ell) \right),$$

where α and β are tuning parameters. The outlier residual cost is high for tracklets whose motions are well-explained by extant labels and low for tracklets with high costs for all labels.

3.4.2 Smoothness Cost

$$\sum_{(\mathcal{p}^i, \mathcal{p}^j) \in \mathcal{N}} \exp(-\omega_{ij}) V(\mathcal{p}^i, \mathcal{p}^j),$$

penalizes neighboring tracklets that do not share the same label by the inverse exponential of their edge cost, ω_{ij} . This encourages a piecewise-rigid solution that is appropriate for most dynamic scenes consisting of contiguous rigid bodies. The cost is a weighted sum of all edges penalized according to

$$V(\mathcal{p}^i, \mathcal{p}^j) := \begin{cases} 1 & \text{if } \ell(\mathcal{p}^i) \neq \ell(\mathcal{p}^j) \\ 0 & \text{otherwise} \end{cases}.$$

This definition of V mirrors the standard Potts model (Boykov et al. 1999). The outlier label smoothness cost is the same as all other labels.

3.4.3 Complexity Cost

$$\sum_{\ell \in \mathcal{L}} \mu_{\ell} \psi(\ell),$$

encourages a compact solution by penalizing the use of each label by some cost. This prioritizes a *minimum description length*, which favors compactness over complexity and avoids overfitting to the data. It is the sum of the per-label cost of each label with nonzero support according to the function,

$$\psi(\ell) := \begin{cases} 1 & \text{if } |\mathcal{P}_{\ell}| > 0 \\ 0 & \text{otherwise} \end{cases}.$$

The cost of using a label, μ_{ℓ} , is an open design decision. All motion labels can be assigned the same cost or label costs can be designed to encourage common motions and penalize kinematically complex motions if prior information about the environment is known, such as the sensor platform or third-party kinematics. A privileged “playbook” of likely motions can also be included in the label set as default options and assigned low costs. The label cost, $\mu_{\mathcal{O}}$, for the outlier label is zero because outliers are assumed to always exist. Significantly decreasing the label weights can lead to an overfit solution, while increasing them can make it difficult to segment small motions.

3.4.4 Label Merging Label merging reduces the total energy in (5) by addressing the model complexity term. Two labels, ℓ and ℓ' , are merged if relabeling all tracklets from one label, $\mathcal{P}_{\ell'}$, to the other label, ℓ , would decrease the total energy. This occurs when the increase in residual error is less than the cost of using the label and any change in smoothness. When more than one merge would reduce the total energy, the merge that results in the greatest decrease in cost is chosen. Merging continues until no more merges would reduce the total energy in (5). The outlier label, $\mathcal{O}$, is excluded from merging.

The label splitting, assignment, and merging stages (Sections 3.3 and 3.4) are iterated until the labels converge or a maximum number of iterations, N_{conv} , have been reached. Label convergence can be defined relative to various criteria, such as the total energy, the change in energy, or the change in the labeling.

3.5 Label Sanitization

Once the label set has converged, the final segmentation is refined to remove noisy tracklets before the final estimation. Individual tracklets whose residual error is greater than the threshold, e_{th} , are relabeled as outliers, regardless of the potential change in energy. Likewise, any label with fewer than a minimum number of support tracklets, N_{th} , or that exists for fewer than a minimum number of frames, K_{th} , is removed and its tracklets are relabeled as outliers. These thresholds remove spurious labels describing small sets of tracklets or brief bulk motions and must be at least 3 points and 2 frames, respectively. This label sanitization provides a consistent final set of tracklets for the batch estimation of each motion.

4 Batch SE (3) Estimation

The $SE(3)$ trajectories of the sensor and each object in the scene are estimated from the segmented tracklets. Batch trajectory estimation techniques traditionally calculate the motion of a sensor relative to a set of points assuming they are static. This is appropriate for egomotion estimation (Section 4.1), but third-party trajectory estimation requires extending these techniques to include the geocentric (i.e., quasi-inertial) frame defined by the egomotion estimation (Section 4.2).

Initially, each segmented motion represents an egomotion trajectory hypothesis, ${}^{\ell}\mathbf{T}_{C_k C_{k-1}}$, under the assumption that the associated tracklets, $\mathcal{P}_{\ell}$, are static. Once the true static label is identified, its corresponding egomotion estimate can be used to estimate the remaining third-party motions in a geocentric frame.

These geocentric trajectories can be estimated without making any assumptions about the object or sensor motion other than rigid motion. The estimation can be made more robust by using more expressive motion models that constrain the trajectories to the real-world kinematics of the scene. Examples of generalized, physically founded $SE(3)$ motion models include the WNOA prior described by Anderson and Barfoot (2015) and the WNOJ prior described by Tang et al. (2019). The WNOA prior extends the pose-only trajectory

state to also include velocity and the WNOJ prior extends it to include both velocity and acceleration.

The applicability of any motion model depends on how well it matches the motions in an environment. The pose-only estimator (Sections 4.1.1 and 4.2.1) can estimate complex rigid motions but can also generate physically implausible motions. The pose-velocity (Sections 4.1.2 and 4.2.2) and pose-velocity-acceleration (Sections 4.1.3 and 4.2.3) estimators penalize deviation from a constant body-centric velocity or acceleration, respectively. They are generally applicable because objects tend to move smoothly through their environment, but they are not robust to motions that deviate significantly from these assumptions. Each estimator brings advantages and disadvantages in the context of the MEP, and this investigation shows how MVO can leverage different estimators in different applications (Sections 6 and 7).

4.1 Egomotion Estimation

The egomotion label, C , may be selected by a variety of heuristics. It can be chosen as the label with the largest support, as in VO, or with the most similar motion to the previous egomotion estimate. This can be combined with external sensors or application-specific heuristics, such as attention masks that prioritize parts of the field of view that usually contain static background objects, to robustly identify the true egomotion label.

Once identified, the point measurements in the egomotion label are assumed to be static. The trajectory is therefore estimated relative to a geocentric frame and inertial motion priors can be used in the estimation.

4.1.1 Pose-Only Estimator A discrete single-motion bundle adjustment (e.g., Barfoot 2017) defines the egomotion state, $\mathbf{x}$, to include both the estimated pose transforms, $\mathcal{T}_C := (\mathbf{T}_{C_k C_1})_{k=2 \dots K}$, and the landmark points, $\{\mathbf{p}_{C_1}^{j_1 C_1}\}_{j=1 \dots |\mathcal{P}_C|}$. The state $\mathbf{x}_{jk} := \{\mathbf{T}_{C_k C_1}, \mathbf{p}_{C_1}^{j_1 C_1}\}$ is defined for each pair of transforms and points in label C , i.e., $\mathbf{T}_{C_k C_1} \in \mathcal{T}_C$ and $\mathbf{p}_{C_1}^{j_1 C_1} \in \mathcal{P}^C$. The batch size, K , is the length of the estimation window, which can either be the entire sequence, or some fixed-length sliding window (Section 5).

Each observation, $\mathbf{y}_{jk}$, of point, $\underline{p}^j$, at pose, $\mathbf{T}_{C_k C_1}$, is modeled as

$$\begin{aligned} \mathbf{y}_{jk} &:= \mathbf{g}(\mathbf{x}_{jk}) + \mathbf{n}_{jk} = \mathbf{s}(\mathbf{z}(\mathbf{x}_{jk})) + \mathbf{n}_{jk} \\ &= \mathbf{s}(\mathbf{T}_{C_k C_1} \mathbf{p}_{C_1}^{j_1 C_1}) + \mathbf{n}_{jk}. \end{aligned} \quad (6)$$

The measurement model, $\mathbf{g}(\cdot)$, encompasses both the motion model, $\mathbf{z}(\cdot)$, which applies the $SE(3)$ motion of the camera, and the sensor model, $\mathbf{s}(\cdot)$. The model assumes additive Gaussian noise, $\mathbf{n}_{jk}$, with zero mean and covariance $\mathbf{R}_{jk}$. The least-squares cost function is defined as

$$J(\mathbf{x}) := \frac{1}{2} \sum_{j=1}^{|\mathcal{P}_C|} \sum_{k=1}^K \mathbf{e}_{y,jk}(\mathbf{x})^T \mathbf{R}_{jk}^{-1} \mathbf{e}_{y,jk}(\mathbf{x}), \quad (7)$$

where,

$$\mathbf{e}_{y,jk}(\mathbf{x}) := \mathbf{y}_{jk} - \mathbf{g}(\mathbf{x}_{jk}),$$

is the residual between the observed and modeled tracklet position.

This cost is linearized about an operating point, $\mathbf{x}_{\text{op}}$, and then minimized using Gauss-Newton. The linearized cost

function is

$$J(\mathbf{x}) \approx J(\mathbf{x}_{\text{op}}) - \mathbf{b}^T \delta \mathbf{x}_{jk} + \frac{1}{2} \delta \mathbf{x}_{jk}^T \mathbf{A} \delta \mathbf{x}_{jk},$$

$$\mathbf{b} = \sum_{j=1}^{|\mathcal{P}_C|} \sum_{k=1}^K \mathbf{P}_{jk}^T \mathbf{G}_{jk}^T \mathbf{R}_{jk}^{-1} \mathbf{e}_{y,jk}(\mathbf{x}_{\text{op}}),$$

$$\mathbf{A} = \sum_{j=1}^{|\mathcal{P}_C|} \sum_{k=1}^K \mathbf{P}_{jk}^T \mathbf{G}_{jk}^T \mathbf{R}_{jk}^{-1} \mathbf{G}_{jk} \mathbf{P}_{jk}.$$

The Jacobian, $\mathbf{G}_{jk}$, of the measurement model, $\mathbf{g}(\cdot)$, at $\mathbf{x}_{jk}$ is given by

$$\mathbf{G}_{jk} := \mathbf{S}_{jk} \mathbf{Z}_{jk}, = \left. \frac{\partial \mathbf{s}}{\partial \mathbf{z}} \right|_{\mathbf{z}(\mathbf{x}_{\text{op}}, jk)} \left. \frac{\partial \mathbf{z}}{\partial \delta \mathbf{x}} \right|_{\mathbf{x}_{\text{op}}, jk}, \quad (8)$$

$$\left. \frac{\partial \mathbf{z}}{\partial \delta \mathbf{x}} \right|_{\mathbf{x}_{\text{op}}, jk} = \left[\left(\mathbf{T}_{\text{op}, C_k C_1} \mathbf{p}_{\text{op}, C_1}^{j_1 C_1} \right)^\odot \quad \mathbf{T}_{\text{op}, C_k C_1} \mathbf{D} \right],$$

$$\mathbf{D} = \begin{bmatrix} \mathbf{1} \\ \mathbf{0}^T \end{bmatrix},$$

where the operator $(\cdot)^\odot : \mathbb{R}^{4 \times 1} \rightarrow \mathbb{R}^{6 \times 6}$ is defined as

$$\mathbf{p}^\odot = \begin{bmatrix} k \mathbf{p} \\ k \end{bmatrix}^\odot := \begin{bmatrix} k \mathbf{1} & -\mathbf{p}^\times \\ \mathbf{0}^T & \mathbf{0}^T \end{bmatrix}$$

and $(\cdot)^\times : \mathbb{R}^{3 \times 1} \rightarrow \mathbb{R}^{3 \times 3}$ is the skew-symmetric matrix operator,

$$\mathbf{p}^\times = \begin{bmatrix} x \\ y \\ z \end{bmatrix}^\times := \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}.$$

The state perturbation, $\delta \mathbf{x}$, consists of the transform perturbations, $\{\delta \boldsymbol{\xi}_k \in \mathbb{R}^6\}$, and landmark perturbations, $\{\delta \boldsymbol{\zeta}_j \in \mathbb{R}^3\}$. The indicator matrix $\mathbf{P}_{jk}$ is defined such that $\delta \mathbf{x}_{jk} = \mathbf{P}_{jk} \delta \mathbf{x}$ is the perturbation of the $\{\mathbf{T}_{\text{op}, C_k C_1}, \mathbf{p}_{\text{op}, C_1}^{j_1 C_1}\}$ pair.

The optimal perturbation, $\delta \mathbf{x}^*$, for minimizing the linearized cost function is the solution to the linear equation $\mathbf{A} \delta \mathbf{x}^* = \mathbf{b}$. Each element of the state is then updated according to

$$\begin{aligned} \mathbf{T}_{\text{op}, C_k C_1} &\leftarrow \exp(\delta \boldsymbol{\xi}_k^*) \mathbf{T}_{\text{op}, C_k C_1} \\ \mathbf{p}_{\text{op}, C_1}^{j_1 C_1} &\leftarrow \mathbf{p}_{\text{op}, C_1}^{j_1 C_1} + \mathbf{D} \delta \boldsymbol{\zeta}_j^*, \end{aligned} \quad (9)$$

where $(\cdot)^\wedge : \mathbb{R}^{6 \times 1} \rightarrow \mathbb{R}^{4 \times 4}$ is the $SE(3)$ lift operator,

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\rho} \\ \phi \end{bmatrix}^\wedge := \begin{bmatrix} \phi^\times & \boldsymbol{\rho} \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (10)$$

The cost function is then relinearized about the updated operating point and the process iterates until convergence. Convergence is user defined and can be a threshold on the number of iterations, the total cost, the change in cost, the magnitude of the update, or other criteria.

4.1.2 Pose-Velocity Estimator The pose-only estimator is applicable to any rigid-body motion, but is not constrained to physically plausible motions. This can be mitigated by constraining the trajectory to a locally constant velocity.

The pose-velocity estimator employs an $SE(3)$ WNOA prior (Anderson and Barfoot 2015),

$$\dot{\boldsymbol{\omega}}_C(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q}_c \delta(t - t')),$$

which defines the trajectory acceleration, $\dot{\boldsymbol{\omega}}_C$, as a continuous zero-mean, white noise Gaussian process with power spectral density matrix, $\mathbf{Q}_c \in \mathbb{R}^{6 \times 6}$. The prior models the velocity as being locally constant, but can vary if there are sufficient observations to support it.

The pose-velocity estimator state, $\mathbf{x}$, comprises the estimated pose transforms, $\mathcal{T}_C$, body-centric velocities, $(\boldsymbol{\omega}_{C_k})_{k=1 \dots K}$, and the labeled landmark points, $\{\mathbf{p}_{C_1}^{j_1 C_1}\}_{j=1 \dots |\mathcal{P}_C|}$. The state $\mathbf{x}_k := \{\mathbf{T}_{C_k C_1}, \boldsymbol{\omega}_{C_k}\}$ is defined as the transform and velocity at each time, t_k , and $\mathbf{x}_{jk} := \{\mathbf{x}_k, \mathbf{p}_{C_1}^{j_1 C_1}\}$ is defined for each pair of time steps and points in the label.

The prior is characterized by the transition function,

$$\Phi(\tau, t_k) := \begin{bmatrix} \mathbf{1} & \Delta t_{\tau, k} \mathbf{1} \\ \mathbf{0} & \mathbf{1} \end{bmatrix}, \quad (11)$$

where $\Delta t_{\tau, k} = \tau - t_k$.

The transition function propagates the local prior state, γ_{C_k} , from time t_k to time τ according to

$$\gamma_{C_k}(\tau) = \Phi(\tau, t_k) \gamma_{C_k}(t_k), \quad (12)$$

where

$$\gamma_{C_k}(\tau) := \begin{bmatrix} \mathbf{T}_{C_k}(\tau) \\ \boldsymbol{\omega}_{C_k}(\tau) \end{bmatrix}.$$

The least-squares estimate is found by minimizing the combined objective function,

$$J(\mathbf{x}) := J_y(\mathbf{x}) + J_p(\mathbf{x}), \quad (13)$$

consisting of the measurement and prior terms. The measurement term, $J_y(\mathbf{x})$, constrains the estimated state by the observations, as given in (7). The prior term, $J_p(\mathbf{x})$, constrains the trajectory and velocity estimates by the constant-velocity assumption,

$$J_p(\mathbf{x}) := \frac{1}{2} \sum_{k=1}^{K-1} \mathbf{e}_{p,k}(\mathbf{x})^T \mathbf{Q}_k(t_{k+1})^{-1} \mathbf{e}_{p,k}(\mathbf{x}), \quad (14)$$

where the error penalizes deviation from the constant-velocity prior defined in (12),

$$\mathbf{e}_{p,k}(\mathbf{x}) := \gamma_{C_k}(t_{k+1}) - \Phi(t_{k+1}, t_k) \gamma_{C_k}(t_k),$$

and the block covariance matrix,

$$\mathbf{Q}_k(\tau) := \begin{bmatrix} \frac{1}{3} \Delta t_{\tau, k}^3 \mathbf{Q}_c & \frac{1}{2} \Delta t_{\tau, k}^2 \mathbf{Q}_c \\ \frac{1}{2} \Delta t_{\tau, k}^2 \mathbf{Q}_c & \Delta t_{\tau, k} \mathbf{Q}_c \end{bmatrix}, \quad (15)$$

has the inverse,

$$\mathbf{Q}_k^{-1}(\tau) := \begin{bmatrix} 12 \Delta t_{\tau, k}^{-3} \mathbf{Q}_c^{-1} & -6 \Delta t_{\tau, k}^{-2} \mathbf{Q}_c^{-1} \\ -6 \Delta t_{\tau, k}^{-2} \mathbf{Q}_c^{-1} & 4 \Delta t_{\tau, k}^{-1} \mathbf{Q}_c^{-1} \end{bmatrix}.$$

The total cost is minimized by linearizing the error about an operating point, $\mathbf{x}_{\text{op}}$,

$$J(\mathbf{x}) \approx J(\mathbf{x}_{\text{op}}) - \mathbf{b}^T \delta \mathbf{x} + \frac{1}{2} \delta \mathbf{x}^T \mathbf{A} \delta \mathbf{x}, \quad (16)$$

where,

$$\begin{aligned} \mathbf{b} &= \sum_{j=1}^{|\mathcal{P}_C|} \sum_{k=1}^K \mathbf{P}_{jk}^T \mathbf{G}_{jk}^T \mathbf{R}_{jk}^{-1} \mathbf{e}_{y,jk}(\mathbf{x}_{\text{op}}) \\ &\quad + \sum_{k=1}^{K-1} \mathbf{P}_k^T \mathbf{E}_k^T \mathbf{Q}_k^{-1} \mathbf{e}_{p,k}(\mathbf{x}_{\text{op}}), \\ \mathbf{A} &= \sum_{j=1}^{|\mathcal{P}_C|} \sum_{k=1}^K \mathbf{P}_{jk}^T \mathbf{G}_{jk}^T \mathbf{R}_{jk}^{-1} \mathbf{G}_{jk} \mathbf{P}_{jk} + \sum_{k=1}^{K-1} \mathbf{P}_k^T \mathbf{E}_k^T \mathbf{Q}_k^{-1} \mathbf{E}_k \mathbf{P}_k, \end{aligned}$$

and $\mathbf{G}_{jk}$ and $\mathbf{E}_k$ are the Jacobians of the measurement and prior error functions, respectively.

The Jacobian of the measurement model, $\mathbf{G}_{jk}$, is defined according to (8) with the updated motion model Jacobian,

$$\left. \frac{\partial \mathbf{z}}{\partial \delta \mathbf{x}} \right|_{\mathbf{x}_{\text{op},jk}} = \begin{bmatrix} (\mathbf{T}_{\text{op},C_k C_1} \mathbf{p}_{j_1 C_1}^{\text{op},C_1})^\odot & \mathbf{0} & \mathbf{T}_{\text{op},C_k C_1} \mathbf{D} \end{bmatrix}.$$

The Jacobian of the prior error function, $\mathbf{E}_{jk}$, is

$$\begin{aligned} \mathbf{E}_k &= \begin{bmatrix} \mathbf{E}_{11} & \Delta t_{k+1,k} \mathbf{1} & \mathbf{E}_{13} & \mathbf{0} \\ \mathbf{E}_{21} & \mathbf{1} & \mathbf{E}_{23} & \mathbf{E}_{24} \end{bmatrix}, \\ \mathbf{E}_{11} &= \mathcal{J}_{C_{k+1}C_k}^{-1} \mathcal{T}_{C_{k+1}C_k}, \\ \mathbf{E}_{13} &= -\mathcal{J}_{C_{k+1}C_k}^{-1}, \\ \mathbf{E}_{21} &= \frac{1}{2} \varpi_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1}C_k}^{-1} \mathcal{T}_{C_{k+1}C_k}, \\ \mathbf{E}_{23} &= -\frac{1}{2} \varpi_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1}C_k}^{-1}, \\ \mathbf{E}_{24} &= -\mathcal{J}_{C_{k+1}C_k}^{-1}. \end{aligned}$$

where the operator $(\cdot)^\wedge : \mathbb{R}^{6 \times 1} \rightarrow \mathbb{R}^{6 \times 6}$ is defined as

$$\boldsymbol{\xi}^\wedge := \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\phi} \end{bmatrix}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho}^\times \\ \mathbf{0} & \boldsymbol{\phi}^\wedge \end{bmatrix},$$

using the $SO(3)$ lift operator, $(\cdot)^\wedge : \mathbb{R}^{3 \times 1} \rightarrow \mathbb{R}^{3 \times 3}$, which is equivalent to the skew-symmetric operator, $(\cdot)^\times$.

The adjoint of $\mathbf{T}_{C_{k+1}C_k}$ is defined as

$$\begin{aligned} \mathcal{T}_{C_{k+1}C_k} &:= \exp(\boldsymbol{\xi}_{C_{k+1}C_k}^\wedge) \\ &= \begin{bmatrix} \mathbf{C}_{C_{k+1}C_k} & (\mathbf{p}_{C_{k-1}}^{C_k C_{k-1}})^\times \mathbf{C}_{C_{k+1}C_k} \\ \mathbf{0} & \mathbf{C}_{C_{k+1}C_k} \end{bmatrix}, \end{aligned}$$

where

$$\boldsymbol{\xi}_{C_{k+1}C_k} = \ln(\mathbf{T}_{C_{k+1}C_k})^\vee,$$

and $(\cdot)^\vee : \mathbb{R}^{4 \times 4} \rightarrow \mathbb{R}^{6 \times 1}$ is the inverse of the $SE(3)$ lift operator in (10). The inverse of the left Jacobian of $SE(3)$, $\mathcal{J}_{C_{k+1}C_k}^{-1} \in \mathbb{R}^{6 \times 6}$, is defined as

$$\begin{aligned} \mathcal{J}_{C_{k+1}C_k}^{-1} &:= \left(\int_0^1 \mathcal{T}_{C_{k+1}C_k}^\alpha d\alpha \right)^{-1} \\ &\approx \mathbf{1} - \frac{1}{2} \boldsymbol{\xi}_{C_{k+1}C_k}^\wedge. \end{aligned}$$

The operating point is perturbed according to the transform perturbations, $\{\delta \boldsymbol{\xi}_k \in \mathbb{R}^6\}$, velocity perturbations $\{\delta \boldsymbol{\varpi}_k \in \mathbb{R}^6\}$, and landmark perturbations, $\{\delta \boldsymbol{\zeta}_j \in \mathbb{R}^3\}$,

which are stacked to form the full state perturbation, $\delta \mathbf{x}$. The indicator matrices $\mathbf{P}_{jk}$ and $\mathbf{P}_k$ are defined such that $\delta \mathbf{x}_{jk} = \mathbf{P}_{jk} \delta \mathbf{x}$ is the perturbation of the $\{\mathbf{T}_{\text{op},C_k C_1}, \boldsymbol{\varpi}_{\text{op},C_k}, \mathbf{p}_{\text{op},C_1}^{j_1 C_1}\}$ tuple and $\delta \mathbf{x}_k = \mathbf{P}_k \delta \mathbf{x}$ is the perturbation of the $\{\mathbf{T}_{\text{op},C_k C_1}, \boldsymbol{\varpi}_{\text{op},C_k}\}$ pair.

The optimal perturbation, $\delta \mathbf{x}^*$, to minimize the linearized cost is the solution to the linear equation, $\mathbf{A} \delta \mathbf{x}^* = \mathbf{b}$. Each element of the operating point is then updated using (9) and

$$\boldsymbol{\varpi}_{C_k} \leftarrow \boldsymbol{\varpi}_{C_k} + \delta \boldsymbol{\varpi}_k^*. \quad (17)$$

The cost function is then relinearized about the updated operating point and the process iterates until a user-defined convergence criterion.

4.1.3 Pose-Velocity-Acceleration Estimator The WNOA prior incentivizes locally-constant velocity, i.e., zero acceleration, which yields more physically plausible velocities, but is less accurate for motions with nonconstant velocity. This limitation is particularly important for objects that change direction, and a prior that can model nonzero accelerations is more representative of real-world motions.

The WNOJ prior models the trajectory *jerk* as a continuous zero-mean, white noise Gaussian process (Tang et al. 2019),

$$\ddot{\boldsymbol{\varpi}}_C(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q}_c \delta(t - t')).$$

This is analogous to the pose-velocity estimator instead with a locally constant *acceleration*. The prior can model nonzero accelerations and is more applicable to real-world motions with objects that change direction.

The pose-velocity-acceleration estimator state, $\mathbf{x}$, comprises the estimated pose transforms, $\mathcal{T}_C$, the body-centric velocities, $(\boldsymbol{\varpi}_{C_k})_{k=1 \dots K}$, the body-centric accelerations, $(\ddot{\boldsymbol{\varpi}}_{C_k})_{k=1 \dots K}$, and the labeled landmark points, $\{\mathbf{p}_{C_1}^{j_1 C_1}\}_{j=1 \dots |\mathcal{P}_C|}$. The state $\mathbf{x}_k := \{\mathbf{T}_{C_k C_1}, \boldsymbol{\varpi}_{C_k}, \ddot{\boldsymbol{\varpi}}_{C_k}\}$ is defined as the transform, velocity, and acceleration at each time, t_k , and $\mathbf{x}_{jk} := \{\mathbf{x}_k, \mathbf{p}_{C_1}^{j_1 C_1}\}$ is defined for each pair of time steps and points in the label.

The estimator proceeds analogously to the pose-velocity estimator using the same combined objective function in (13) with an expanded state transition function,

$$\Phi(\tau, t_k) := \begin{bmatrix} \mathbf{1} & (\Delta t_{\tau,k}) \mathbf{1} & \frac{1}{2} (\Delta t_{\tau,k})^2 \mathbf{1} \\ \mathbf{0} & \mathbf{1} & (\Delta t_{\tau,k}) \mathbf{1} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}. \quad (18)$$

The expanded block covariance matrix,

$$\mathbf{Q}_k(\tau) := \begin{bmatrix} \frac{1}{20} \Delta t_{\tau,k}^5 \mathbf{Q}_c & \frac{1}{8} \Delta t_{\tau,k}^4 \mathbf{Q}_c & \frac{1}{6} \Delta t_{\tau,k}^3 \mathbf{Q}_c \\ \frac{1}{8} \Delta t_{\tau,k}^4 \mathbf{Q}_c & \frac{1}{3} \Delta t_{\tau,k}^3 \mathbf{Q}_c & \frac{1}{2} \Delta t_{\tau,k}^2 \mathbf{Q}_c \\ \frac{1}{6} \Delta t_{\tau,k}^3 \mathbf{Q}_c & \frac{1}{2} \Delta t_{\tau,k}^2 \mathbf{Q}_c & \Delta t_{\tau,k} \mathbf{Q}_c \end{bmatrix}, \quad (19)$$

has the inverse,

$$\mathbf{Q}_k^{-1}(\tau) := \begin{bmatrix} 720 \Delta t_{\tau,k}^{-5} \mathbf{Q}_c^{-1} & -360 \Delta t_{\tau,k}^{-4} \mathbf{Q}_c^{-1} & 60 \Delta t_{\tau,k}^{-3} \mathbf{Q}_c^{-1} \\ -360 \Delta t_{\tau,k}^{-4} \mathbf{Q}_c^{-1} & 192 \Delta t_{\tau,k}^{-3} \mathbf{Q}_c^{-1} & -36 \Delta t_{\tau,k}^{-2} \mathbf{Q}_c^{-1} \\ 60 \Delta t_{\tau,k}^{-3} \mathbf{Q}_c^{-1} & -36 \Delta t_{\tau,k}^{-2} \mathbf{Q}_c^{-1} & 9 \Delta t_{\tau,k}^{-1} \mathbf{Q}_c^{-1} \end{bmatrix}.$$

The total cost is again minimized by linearizing the error about an operating point, $\mathbf{x}_{\text{op}}$. The linearized form is identical

to (16) where the Jacobian of measurement model, $\mathbf{G}_{jk}$, in (8) is calculated with the updated motion model Jacobian,

$$\frac{\partial \mathbf{z}}{\partial \delta \mathbf{x}} \bigg|_{\mathbf{x}_{\text{op},jk}} = \begin{bmatrix} (\mathbf{T}_{\text{op},C_k C_1} \mathbf{p}_{\text{op},C_1}^{j_1 C_1})^\odot & \mathbf{0} & \mathbf{0} & \mathbf{T}_{\text{op},C_k C_1} \mathbf{D} \end{bmatrix},$$

and the Jacobian of the prior error function is given as

$$\mathbf{E}_k = \begin{bmatrix} \mathbf{E}_{11} & \Delta t_{k+1,k} \mathbf{1} & \frac{1}{2} \Delta t_{k+1,k}^2 \mathbf{1} & \mathbf{E}_{13} & \mathbf{0} & \mathbf{0} \\ \mathbf{E}_{21} & \mathbf{1} & \Delta t_{k+1,k} \mathbf{1} & \mathbf{E}_{24} & \mathbf{E}_{25} & \mathbf{0} \\ \mathbf{E}_{31} & \mathbf{0} & \mathbf{1} & \mathbf{E}_{34} & \mathbf{E}_{35} & \mathbf{E}_{36} \end{bmatrix},$$

$$\mathbf{E}_{11} = \mathcal{J}_{C_{k+1} C_k}^{-1} \mathcal{T}_{C_{k+1} C_k},$$

$$\mathbf{E}_{13} = -\mathcal{J}_{C_{k+1} C_k}^{-1},$$

$$\mathbf{E}_{21} = \frac{1}{2} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1} \mathcal{T}_{C_{k+1} C_k},$$

$$\mathbf{E}_{24} = -\frac{1}{2} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1},$$

$$\mathbf{E}_{25} = -\mathcal{J}_{C_{k+1} C_k}^{-1},$$

$$\mathbf{E}_{31} = \frac{1}{4} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1} \mathcal{T}_{C_{k+1} C_k} + \frac{1}{2} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1} \mathcal{T}_{C_{k+1} C_k},$$

$$\mathbf{E}_{34} = \frac{1}{4} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1} + \frac{1}{2} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1},$$

$$\mathbf{E}_{35} = \frac{1}{2} \left(\mathcal{J}_{C_{k+1} C_k}^{-1} \mathcal{W}_{C_{k+1}}^\wedge \right)^\wedge - \frac{1}{2} \mathcal{W}_{C_{k+1}}^\wedge \mathcal{J}_{C_{k+1} C_k}^{-1},$$

$$\mathbf{E}_{36} = \mathcal{J}_{C_{k+1} C_k}^{-1}.$$

The operating point is perturbed according to the transform perturbations, $\{\delta \xi_k \in \mathbb{R}^6\}$, velocity perturbations $\{\delta \mathbf{w}_k \in \mathbb{R}^6\}$, acceleration perturbations $\{\delta \mathbf{a}_k \in \mathbb{R}^6\}$, and landmark perturbations, $\{\delta \zeta_j \in \mathbb{R}^3\}$, which are stacked to form the full state perturbation, $\delta \mathbf{x}$. The indicator matrices $\mathbf{P}_{jk}$ and $\mathbf{P}_k$ are defined such that $\delta \mathbf{x}_{jk} = \mathbf{P}_{jk} \delta \mathbf{x}$ is the perturbation of the $\{\mathbf{T}_{\text{op},C_k C_1}, \mathcal{W}_{\text{op},C_k}, \mathcal{W}_{C_k}, \mathbf{p}_{\text{op},C_1}^{j_1 C_1}\}$ tuple and $\delta \mathbf{x}_k = \mathbf{P}_k \delta \mathbf{x}$ is the perturbation of the $\{\mathbf{T}_{\text{op},C_k C_1}, \mathcal{W}_{\text{op},C_k}, \mathcal{W}_{C_k}\}$ tuple.

The optimal perturbation, $\delta \mathbf{x}^*$, to minimize the linearized cost is the solution to the linear equation $\mathbf{A} \delta \mathbf{x}^* = \mathbf{b}$. The operating point is then updated using (9), (17), and

$$\mathcal{W}_{C_k} \leftarrow \mathcal{W}_{C_k} + \delta \mathbf{w}_k^*,$$

and the cost is relinearized about the updated operating point until a user-defined convergence criterion.

4.2 Geocentric Third-Party Estimation

Egomotion estimation techniques must be adapted to estimate the geocentric trajectories of dynamic objects that invalidate the static tracklet assumption. Initial versions of the ideas in Sections 4.2.1 and 4.2.2 were first published in Judd et al. (2018a) and Judd and Gammell (2020), respectively, and they are further developed here. The ideas in Section 4.2.3 are presented here for the first time.

Pose-only estimates (Section 4.1.1) in an egocentric frame can be directly converted to a geocentric frame (Section 4.2.1). Inertial priors, such as the WNOA (Section 4.1.2) and WNOJ (Section 4.1.3) priors, are more complicated because bodies moving with a constant motion relative to a common inertial reference frame do not have constant motion relative to *each*

other. Applying these motion priors to third-party trajectories requires rederiving the motion model in (6) and its associated Jacobians in (8) (Sections 4.2.2 and 4.2.3).

4.2.1 Pose-Only Estimator $SE(3)$ transforms initially estimated using egomotion estimation techniques (Section 4.1.1) represent egomotion hypotheses calculated from the assumption that observed points are static. When the tracklets are moving, these estimates are the inverse of the objects' motion relative to the sensor, i.e., the inverse of their egocentric motion.

Egomotion hypothesis trajectories, ${}^\ell \mathcal{T}_C$, can be transformed into true third-party geocentric trajectories by compensating for the identified egomotion estimate, $\mathbf{T}_{C_k C_1}$, via

$$\forall \ell \in \mathcal{L} \setminus C, \quad \mathbf{T}_{\ell_k \ell_1} = \mathbf{F}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1} {}^\ell \mathbf{T}_{C_k C_1}^{-1} \mathbf{T}_{C_k C_1} \mathbf{T}_{\ell_1 C_1}^{-1}, \quad (20)$$

where $\mathbf{F}_{\ell_k \ell_1}$ is the object deformation matrix and is assumed to be identity, (i.e., rigid-body motion).

The initial sensor-to-object transform, $\mathbf{T}_{\ell_1 C_1}$, relates the object frame from the sensor frame and is calculated blockwise from the initial translation and rotation. The initial translation, $\mathbf{p}_{\ell_1}^{C_1 \ell_1}$, of the object frame is assigned to the observed centroid of the labeled points, $\mathcal{P}_\ell$, projected into the first observed frame,

$$\begin{aligned} \bar{\mathbf{p}}_{C_1}^{\ell_1 C_1} &= \frac{1}{|\mathcal{P}_\ell|} \sum_{j=1}^{|\mathcal{P}_\ell|} {}^\ell \mathbf{T}_{C_{t_j} C_1}^{-1} \mathbf{p}_{C_{t_j}}^{j_{t_j} C_{t_j}}, \\ \mathbf{p}_{\ell_1}^{C_1 \ell_1} &= -\mathbf{C}_{\ell_1 C_1} \bar{\mathbf{p}}_{C_1}^{\ell_1 C_1}, \end{aligned} \quad (21)$$

where t_j is the first frame where the tracklet is observed. The initial rotation, $\mathbf{C}_{\ell_1 C_1}$, is the initial frame orientation, which may be calculated from semantic information or be user specified (e.g., identity).

Note that the egomotion estimate is only needed to convert third-party motions from the egocentric to geocentric frames and all motions are batch estimated first as egomotion hypotheses (Section 4.1.1) before identifying the egomotion label. This means motion-based heuristics and other application-specific information can be used to identify the egomotion before converting the remaining third-party motions to geocentric estimates.

4.2.2 Pose-Velocity Estimator The WNOA prior is defined in an inertial frame, so third-party motions are estimated after identifying the egomotion. The techniques used to estimate the egomotion (Section 4.1.2) must be adapted to estimate third-party motions in a geocentric frame.

The transform model, $\mathbf{z}$, in (6) is adjusted to transform egocentrically observed points through a geocentrically estimated state,

$$\mathbf{z}'(\mathbf{x}_{jk}) := \mathbf{T}_{C_k C_1} \mathbf{T}_{\ell_1 C_1}^{-1} \mathbf{T}_{\ell_k \ell_1}^{-1} \mathbf{F}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1} \mathbf{p}_{C_1}^{j_1 C_1}, \quad (22)$$

where $\mathbf{F}_{\ell_k \ell_1}$ is the object deformation matrix (identity for rigid bodies), and $\mathbf{T}_{C_k C_1}$ is the identified egomotion (Section 4.1). The motion model component of the measurement

Jacobian, $\mathbf{G}_{jk}$, in (8) is given by the block row vector

$$\begin{aligned} \left. \frac{\partial \mathbf{z}'}{\partial \delta \mathbf{x}} \right|_{\mathbf{x}_{\text{op},jk}} &= [\mathbf{Z}_1 \quad \mathbf{0} \quad \mathbf{Z}_3], \\ \mathbf{Z}_1 &= -\mathbf{T}_{C_k C_1} \mathbf{T}_{\ell_1 C_1}^{-1} \mathbf{T}_{\text{op}, \ell_k \ell_1}^{-1} \left(\mathbf{F}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1} \mathbf{p}_{\text{op}, C_1}^{j_1 C_1} \right)^{\odot}, \\ \mathbf{Z}_3 &= \mathbf{T}_{C_k C_1} \mathbf{T}_{\ell_1 C_1}^{-1} \mathbf{T}_{\text{op}, \ell_k \ell_1}^{-1} \mathbf{F}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1} \mathbf{D}. \end{aligned} \quad (23)$$

The prior cost is the same as in (14). The linearized cost in (16) is then used to estimate the geocentric trajectory of every third-party motion in the scene.

Note that the egomotion must be estimated before the third-party motions, so motion-based heuristics cannot be used to help identify the egomotion label as in pose-only estimation.

4.2.3 Pose-Velocity-Acceleration Estimator The WNOJ prior is also defined in an inertial frame and is adapted using a similar procedure to the pose-velocity estimator (Section 4.2.2).

The transform model, $\mathbf{z}$, in (6) takes the same form as (22). The motion model component of the measurement Jacobian, $\mathbf{G}_{jk}$, in (8) is given by the block row vector

$$\left. \frac{\partial \mathbf{z}'}{\partial \delta \mathbf{x}} \right|_{\mathbf{x}_{\text{op},jk}} = [\mathbf{Z}_1 \quad \mathbf{0} \quad \mathbf{0} \quad \mathbf{Z}_3],$$

where $\mathbf{Z}_1$ and $\mathbf{Z}_3$ are defined in (23). The linearized cost takes the same form as (16) and uses the same prior cost term as (14).

Note again that the egomotion must be estimated before the third-party motions and therefore motion-based heuristics cannot be used to identify the egomotion label.

5 Sliding-Window Multimotion Estimation

Sections 3 and 4 present a full-batch implementation of MVO. Full-batch estimation requires all observations of a given data segment at once but makes motion segmentation and estimation more accurate because motions are more distinct over longer periods of time. It can also reason about temporary occlusions after they occur, making it easier to determine when an object becomes occluded or unoccluded, as well as track motions through those occlusions. Full-batch estimation is computationally expensive and is often used in offline estimation systems.

MVO can also be implemented as an online sliding-window pipeline that processes data as it is observed and is less computationally expensive than a full-batch implementation. Sliding-window estimation uses smaller batches of the K -most-recent observation frames, and maintaining consistent trajectories over consecutive batches presents several challenges.

Motion segmentation is more difficult in a sliding-window pipeline because independent motions are often less distinct over short timescales. Motions must be associated across estimation windows to maintain label consistency because motions in separate windows are not guaranteed to be assigned the same label (Section 5.1). Unlike full-batch estimation, sliding-window estimation must reason about temporary occlusions as they occur. This requires motions to be extrapolated from, and interpolated between, direct observations (Section 5.2). Detecting when an object becomes

unoccluded requires determining when a newly segmented motion is actually the reappearance of a previously occluded motion, i.e., *motion closure* (Section 5.3). Initial versions of these ideas were first presented in Judd and Gammell (2019b) and Judd and Gammell (2020), and they are further developed and fully detailed for the first time here.

5.1 Label Consistency

Sliding-window estimation requires consistently tracking motions across multiple windows. This involves associating motions in consecutive estimation windows to form a single trajectory over the entirety of the data.

Each new window associates currently estimated motion labels, ℓ_k , with previously estimated motion labels, ℓ'_{k-1} , based on the overlap in their labeled tracklets,

$$\ell_k := \arg \max_{\ell'_{k-1}} |\mathcal{P}_{\ell_k} \cup \mathcal{P}_{\ell'_{k-1}}|. \quad (24)$$

Newly estimated motions without sufficient overlap with a previous motion are tracked as new object motions. Other motion- or appearance-based heuristics could also be employed to make this association more robust.

The labeling and estimates from the current window are used to initialize the segmentation and egomotion hypotheses in the next window. Any current tracklets that also exist in the next window retain their labels, and the current trajectory estimates are extrapolated forward to the next time stamp to initialize the label set. This is done by extrapolating the geocentric estimates, $\mathbf{T}_{\ell_k \ell_1}$, forward and then converting them into egomotion hypotheses by rearranging (20),

$${}^{\ell} \mathbf{T}_{C_k C_1} = \mathbf{T}_{C_k C_1} \mathbf{T}_{\ell_1 C_1}^{-1} \mathbf{T}_{\ell_k \ell_1}^{-1} \mathbf{F}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1}. \quad (25)$$

Consistently observed motion labels are straightforward to propagate (Section 5.2), but this is more difficult in the presence of occlusion (Section 5.3).

5.2 Extrapolation and Interpolation

Unobserved motions can be estimated by modeling the expected motions of previous observations. This is less accurate than direct estimation, but can extend existing motions to initialize the next set of motion estimates as well as estimate motions through temporary occlusions. The accuracy of these estimates depends on the fidelity of the motion model to the true motion of the object, and inferring the motion of an object between two states, i.e., interpolation, is more accurate than from a single state, i.e., extrapolation.

Pose extrapolation and interpolation require an estimate of the change in pose over time, i.e., the velocity. In the pose-only estimator (Sections 4.1.1 and 4.2.1), the velocity is discretely calculated from the relative transform between the final two poses,

$$\varpi_{\ell_k} := \frac{1}{\Delta t_{k,k-1}} \xi_{\ell_{k+1} \ell_k}.$$

The body-centric velocity is continuously estimated in the pose-velocity (Sections 4.1.2 and 4.2.2) and pose-velocity-acceleration estimators (Sections 4.1.3 and 4.2.3).

The pose-velocity-acceleration estimator also directly estimates the body-centric acceleration, which can be used for extrapolation and interpolation. This term is zero for the pose-only and pose-velocity estimators, i.e., constant-velocity extrapolation.

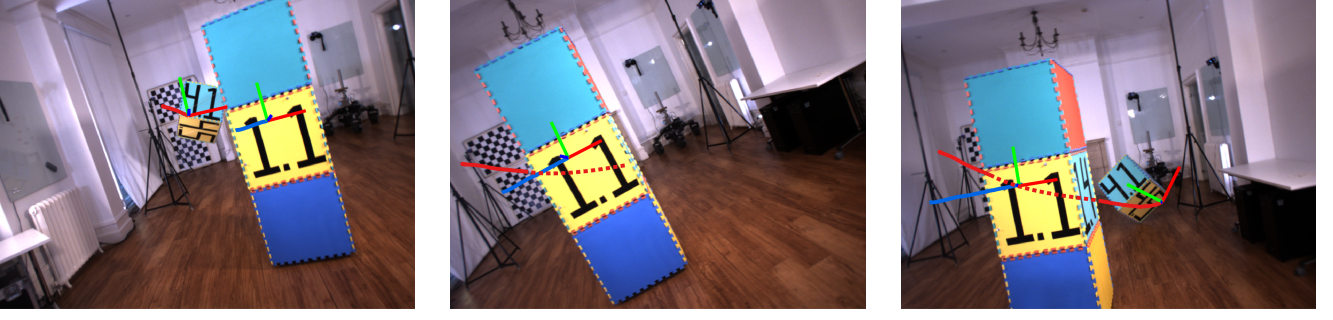


Figure 4. A demonstration of the *motion closure* procedure showing trajectory estimates produced before (left), during (center), and after (right) an occlusion in the `occlusion_2_unconstrained` segment of the OMD. The trajectory of the swinging block (4, red) is directly estimated when it is visible and is extrapolated using the constant-velocity motion prior (dashed line) when the block is occluded by the moving tower (1, blue). When the block becomes unoccluded, it is rediscovered through motion closure and the estimates are interpolated to match the directly estimated trajectory.

5.2.1 Extrapolation The local state, γ_{ℓ_k} , of each estimator is extrapolated (Anderson and Barfoot 2015; Tang et al. 2019) via the transition function (18) to time τ ,

$$\tilde{\gamma}_{\ell_k}(\tau) = \Phi(\tau, t_k) \gamma_{\ell_k}(t_k) = \Phi(\tau, t_k) \begin{bmatrix} \mathbf{0} \\ \tilde{\omega}_{\ell_k} \\ \tilde{\dot{\omega}}_{\ell_k} \end{bmatrix}, \quad (26)$$

where $\tilde{\cdot}$ denotes an extrapolated or interpolated value. Note that when the body-centric acceleration, $\tilde{\omega}_{\ell_k}$, is zero, as with the pose-only and the pose-velocity estimators, (26) is mathematically equivalent to (11) and (12) with extra block zeros.

The extrapolated local state is transformed to the global pose, velocity, and acceleration state via

$$\begin{aligned} \mathbf{T}_{\tilde{\ell}}(\tau) &= \exp\left(\left(\begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \end{bmatrix} \tilde{\gamma}_{\ell_k}(\tau)\right)^\vee\right) \mathbf{T}_{\ell}(t_k), \\ \tilde{\omega}_{\tilde{\ell}}(\tau) &= \begin{bmatrix} \mathbf{0} & \mathbf{1} & \mathbf{0} \end{bmatrix} \tilde{\gamma}_{\ell_k}(\tau), \\ \tilde{\dot{\omega}}_{\tilde{\ell}}(\tau) &= \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix} \tilde{\gamma}_{\ell_k}(\tau), \end{aligned} \quad (27)$$

where the $\tilde{\omega}_{\tilde{\ell}}(\tau)$ term is omitted for the pose-only estimator and the $\tilde{\dot{\omega}}_{\tilde{\ell}}(\tau)$ term is omitted for both the pose-only and pose-velocity estimators.

Estimates can be extrapolated forward or backward in time. The extrapolation accuracy will degrade over time, especially if the true motion exhibits significant changes in velocity or acceleration.

5.2.2 Interpolation When the unobserved states are constrained by direct estimates on both sides, they can be estimated through interpolation. This is useful when tracking is resumed after an occlusion. The accuracy of the previously extrapolated estimates is improved because interpolation considers the directly observed data both before and after the occlusion.

The pose-only state is linearly interpolated to an intermediate time, $t_j < \tau < t_k$, via

$$\begin{aligned} \mathbf{T}_{\tilde{\ell}}(\tau) &= \exp(\xi_{\tau j}^\wedge) \mathbf{T}_{\ell}(t_j) \\ \xi_{\tau j} &= \frac{\Delta t_{\tau j}}{\Delta t_{kj}} \xi_{\ell_k \ell_j}, \end{aligned}$$

which enforces a strict constant-velocity assumption.

The pose-velocity and pose-velocity-acceleration states are interpolated (Anderson and Barfoot 2015; Tang et al. 2019)

via

$$\tilde{\gamma}_{\ell_j}(\tau) = \Lambda(\tau) \gamma_{\ell_j}(t_j) + \Omega(\tau) \gamma_{\ell_j}(t_k),$$

where

$$\begin{aligned} \gamma_{\ell_j}(t_k) &= \begin{bmatrix} \xi_{\ell_k \ell_j} \\ \mathcal{J}_{\ell_k \ell_j}^{-1} \tilde{\omega}_{\ell_k} \\ -\frac{1}{2} \left(\mathcal{J}_{\ell_k \ell_j}^{-1} \tilde{\omega}_{\ell_k} \right)^\wedge \tilde{\omega}_{\ell_k} + \mathcal{J}_{\ell_k \ell_j}^{-1} \tilde{\dot{\omega}}_{\ell_k} \end{bmatrix}, \\ \Lambda(\tau) &= \Phi(\tau, t_j) - \Omega(\tau) \Phi(t_k, t_j), \\ \Omega(\tau) &= \mathbf{Q}_j(\tau) \Phi(t_k, \tau)^T \mathbf{Q}_j(t_k)^{-1}. \end{aligned}$$

The block covariance matrix, $\mathbf{Q}_j(\tau)$, is given by (15) for the WNOA prior and (19) for the WNOJ prior. The interpolated state is then transformed to the global state via (27).

Calculating the interpolation involves estimates both before and after the occlusion. This requires recognizing when a newly discovered motion can be explained by the reappearance of a previously occluded object, i.e., *motion closure* (Section 5.3).

5.3 Motion Closure

Online, sliding-window estimation requires actively recognizing when objects become occluded and unoccluded. While most existing tracking techniques address occlusions with appearance-based object detectors and simple motion models, MVO tracks arbitrary objects through occlusions using motion-based tracking metrics (Fig. 4). This *motion closure* recognizes when a motion becomes unoccluded (Section 5.3.1), corrects the extrapolated pose using information from the new trajectory (Section 5.3.2), and updates the previously extrapolated estimates through interpolation (Section 5.3.3).

5.3.1 Recognizing Previously Occluded Motions

Motions are reacquired by comparing a newly discovered motion, ℓ' , to an extrapolated previously observed motion, $\tilde{\ell}$, at a time, t_k . The two motions are considered the same and *closed* if a weighted combination of the difference in their positions, $\mathbf{p}_{C_k}^{\ell' C_k}$ and $\mathbf{p}_{C_k}^{\tilde{\ell} C_k}$, and velocities, $\tilde{\omega}_{\ell'_k}$ and $\tilde{\omega}_{\tilde{\ell}_k}$, is less than a user-defined threshold, ϵ_{mc} ,

$$\begin{aligned} \lambda_{mc} \left\| \mathbf{p}_{C_k}^{\tilde{\ell} C_k} - \mathbf{p}_{C_k}^{\ell' C_k} \right\| + \\ (1 - \lambda_{mc}) \left\| \tilde{\omega}_{\tilde{\ell}_k} - \mathcal{T}_{\tilde{\ell}_k \ell'_k} \tilde{\omega}_{\ell'_k} \right\| < \epsilon_{mc}. \end{aligned}$$

The proportionality term, λ_{mc} , governs how much the difference in position and the difference in velocity each contribute to whether the motions are closed.

The position of the newly discovered motion, $\mathbf{p}_{C_k}^{\ell'_k C_k}$, is calculated from the centroid of labeled points in the first frame the motion was observed as in (21). The position of the extrapolated motion, $\mathbf{p}_{C_k}^{\tilde{\ell}_k C_k}$, can be found from the extrapolated transform, $\mathbf{T}_{\tilde{\ell}_k C_k}$, as in (1) and (2).

The body-centric velocity of the newly discovered motion, $\boldsymbol{\varpi}_{\ell'_k}$, is calculated directly by the estimator (Section 4) and compared to the extrapolated velocity, $\boldsymbol{\varpi}_{\tilde{\ell}_k}$. This comparison is omitted for the pose-only estimator.

This newly closed trajectory is then used to correct the extrapolated pose (Section 5.3.2).

5.3.2 Correcting Occluded States The newly closed motion is adjusted to create a consistent, smooth trajectory. Information from both the extrapolated and newly discovered trajectories is used to correct the extrapolated states from the start of the occlusion to the time of closure, t_k .

The final corrected pose, $\mathbf{T}_{\ell_k \ell_1}$, is calculated by correcting the extrapolated pose, $\mathbf{T}_{\tilde{\ell}_k \ell_1}$, to the same location as the newly discovered pose, $\mathbf{T}_{\ell'_k \ell_1}$, using the correction transform, $\mathbf{T}_{\ell_k \tilde{\ell}_k}$. This correction is calculated blockwise from the translation and rotation components as in (1).

The translation component of the correction transform, $\mathbf{p}_{\ell_k \tilde{\ell}_k}^{\tilde{\ell}_k \ell_k}$, is the difference between the observed and extrapolated centroids, $\mathbf{p}_{C_k}^{\ell'_k C_k}$ and $\mathbf{p}_{C_k}^{\tilde{\ell}_k C_k}$, such that

$$\begin{aligned} \mathbf{p}_{\ell_k \tilde{\ell}_k}^{\tilde{\ell}_k \ell_k} &= \mathbf{C}_{\ell_k C_k} \mathbf{p}_{C_k}^{\tilde{\ell}_k \ell_k} = \mathbf{C}_{\ell_k C_k} \left(\mathbf{p}_{C_k}^{\tilde{\ell}_k \ell'_k} + \mathbf{p}_{C_k}^{\ell'_k \ell_k} \right) \\ &= \mathbf{C}_{\ell_k \tilde{\ell}_k} \mathbf{C}_{\tilde{\ell}_k C_k} \left(\mathbf{p}_{C_k}^{\tilde{\ell}_k C_k} - \mathbf{p}_{C_k}^{\ell'_k C_k} + \mathbf{p}_{C_k}^{\ell'_k \ell_k} \right), \end{aligned}$$

where $\mathbf{p}_{C_k}^{\ell'_k \ell_k} = \mathbf{0}$ because the newly observed centroid is taken as the position of the final corrected pose.

The extrapolated camera-to-object rotation, $\mathbf{C}_{\tilde{\ell}_k C_k}$, comes blockwise from

$$\mathbf{T}_{\ell_k C_k} = \mathbf{T}_{\tilde{\ell}_k \ell_1} \mathbf{T}_{\ell_1 C_1} \mathbf{T}_{C_k C_1}^{-1}$$

using (1), where $\mathcal{F}_{\ell_1}$ is the object frame at the start of estimation, which is not the same as the start of the current sliding window.

The rotation component of the correction transform, $\mathbf{C}_{\ell_k \tilde{\ell}_k}$, is not estimated and therefore taken as identity, which makes the rotation of the final corrected pose, $\mathbf{C}_{\ell_k \ell_1}$, equivalent to the rotation of extrapolated pose, $\mathbf{C}_{\tilde{\ell}_k \ell_1}$. Appearance-based methods could instead be used to estimate absolute object orientation after occlusion.

5.3.3 Updating Closed Trajectories The previously estimated trajectory is updated to incorporate the motion closure. The correction transform, $\mathbf{T}_{\ell_k \tilde{\ell}_k}$, is applied to the previously extrapolated state at the time of the closure, $\mathbf{T}_{\tilde{\ell}_k \ell_1}$, to give the final corrected pose,

$$\mathbf{T}_{\ell_k \ell_1} = \mathbf{T}_{\ell_k \tilde{\ell}_k} \mathbf{T}_{\tilde{\ell}_k \ell_1}.$$

The occluded states between the start of the occlusion and the corrected closure state, $\mathbf{T}_{\ell_k \ell_1}$, are then updated via interpolation (Section 5.2.2).

Table 1. MVO Parameters used in the OMD and KITTI experiments.

Parameter	OMD	KITTI
Window length (K)	8	8
Graph neighbors (k)	4	4
RANSAC threshold (ϵ_{th})	4	6
RANSAC iterations (N_{RANSAC})	100	100
Outlier cost (α)	100	100
Outlier cost (β)	5	5
Assignment proportionality (λ_{sm})	0.5	0.5
Per-label cost (μ_ℓ)	1000	1000
Min support size (N_{th})	20	20
Min trajectory length (K_{th})	3	3
Motion closure threshold (ϵ_{mc})	3	6
Motion closure prop. (λ_{mc})	0.25	0.25
Convergence iterations (N_{conv})	3	3

The states after the closure, $\tau > t_k$, are reestimated directly from measurements relative to the corrected pose, $\mathbf{T}_{\ell_k \ell_1}$. The reestimate follows Section 4.2 with an estimation window starting at t_k , and the associated initial sensor-to-object transform,

$$\mathbf{T}_{\ell_k C_k} = \mathbf{T}_{\ell_k \ell_1} \mathbf{T}_{\ell_1 C_1} \mathbf{T}_{C_k C_1}^{-1},$$

is used as the initial sensor-to-object transform (i.e., $\mathbf{T}_{\ell_1 C_1}$ in (20) and (22)) and the above $\mathbf{T}_{\ell_1 C_1}$ is the original sensor-to-object transform at the start of estimation, found using (21). This uses both the past knowledge and current observations to estimate a smooth motion throughout the occlusion and maintain motion consistency for the entire trajectory.

6 Evaluation

The performance of MVO is evaluated on real-world stereo data from the OMD (Judd and Gammell 2019a) and the KITTI dataset (Geiger et al. 2012). It is evaluated both quantitatively and qualitatively and the accuracy of the different estimators is discussed. MVO is also compared to VDO-SLAM (Zhang et al. 2020b), a dynamic SLAM system that also estimates the $SE(3)$ motion of all objects in the scene, which has reported better performance than CubeSLAM (Yang and Scherer 2019) and ClusterVO (Huang et al. 2020).

Before evaluation, each estimated trajectory must first be calibrated to the ground-truth trajectory frame (Section 6.1). The accuracy of MVO and VDO-SLAM are evaluated using two metrics that give different context to their performance (Section 6.2).

The OMD contains several challenging multimotion scenes with ground-truth trajectory data for each motion in the scene. The `swinging_4_unconstrained` segment of the OMD directly tests the ability to simultaneously segment and estimate multiple independent motions in a scene (Section 6.3). The `occlusion_2_unconstrained` segment exhibits significant occlusion and is used to evaluate the applicability of the motion priors for extrapolation and interpolation (Section 6.4).

The KITTI dataset is a driving dataset that includes ground-truth GPS/INS data for the sensor platform, but no third-party motion data. While it does not allow for the same

quantitative evaluation as the OMD, it is a commonly used benchmark for evaluating navigation algorithms and the segments Drive 0005 and Odometry 03 are used here to qualitatively evaluate performance in real-world driving scenarios (Section 6.5).

The MVO parameters used in the experiments are given in Table 1. Feature detection and matching were performed using LIBVISO2 (Geiger et al. 2011) and the Gauss-Newton minimization was performed with Ceres (Agarwal et al. —) using analytical derivatives (Section 4). Note that the significant differences in sensor and depth of field between the OMD segments and the KITTI segments required slightly different values for two parameters. These parameter tunings would each be appropriate across their respective datasets.

6.1 Error Calibration

Calculating the error between an estimate and the ground truth at a time, t_k , requires calibrating the trajectories at some earlier reference time, $t_j < t_k$. This makes the calibration dependent on the chosen reference time.

The calibration depends on the initial transform between the estimated and ground truth object frames,

$$\mathbf{T}_{\ell_1^{\text{GT}} \ell_1} = \mathbf{T}_{\ell_1^{\text{GT}} A_1^{\text{GT}}} \mathbf{T}_{A_1^{\text{GT}} C} \mathbf{T}_{\ell_1 C_1}^{-1}.$$

where $\mathbf{T}_{A_1^{\text{GT}} C}$ is the transform relating the ground-truth sensor apparatus frame from the left camera frame. The transform, $\mathbf{T}_{A_1^{\text{GT}} C}$, is static and provided in each dataset. The estimated object frame is assigned by each estimator and is arbitrary relative to the ground-truth object frame.

The calibration at any future time is given by propagating the initial calibration forward,

$$\mathbf{T}_{\ell_j^{\text{GT}} \ell_j} = \mathbf{T}_{\ell_j^{\text{GT}} \ell_1^{\text{GT}}} \mathbf{T}_{\ell_1^{\text{GT}} \ell_1} \mathbf{T}_{\ell_1 \ell_j}^{-1}, \quad (28)$$

where

$$\mathbf{T}_{\ell_j \ell_1} = \mathbf{F}_{\ell_j \ell_1} \mathbf{T}_{\ell_1^{\text{GT}} \ell_1}^{-1} \mathbf{F}_{\ell_j^{\text{GT}} \ell_1}^{-1} \mathbf{T}_{\ell_j^{\text{GT}} \ell_1^{\text{GT}}} \mathbf{T}_{\ell_1^{\text{GT}} \ell_1},$$

is the ground-truth object motion expressed in the estimated frame, and $\mathbf{F}$ is again the body deformation matrix and assumed to be identity.

6.2 Error Metrics

The accuracy of the estimation techniques is assessed using two $SE(3)$ error metrics. Global odometric error measures the estimation accuracy of the entire trajectory to a given point. Relative RMS error measures the error over consecutive time steps. Both errors provide useful context for quantifying the accuracy of the trajectory estimation.

Each metric is defined using the error between the estimated motion, $\mathbf{T}_{\ell_k \ell_j}$, and the ground-truth motion, $\mathbf{T}_{\ell_k^{\text{GT}} \ell_j^{\text{GT}}}$, at any time t_k and calibrated via the pose at any time t_j , given as

$$\text{err}_\ell(t_j, t_k) := \mathbf{F}_{\ell_k^{\text{GT}} \ell_j^{\text{GT}}} \mathbf{T}_{\ell_j^{\text{GT}} \ell_j} \mathbf{F}_{\ell_k \ell_j}^{-1} \mathbf{T}_{\ell_k \ell_j} \mathbf{T}_{\ell_j^{\text{GT}} \ell_j}^{-1} \mathbf{T}_{\ell_k^{\text{GT}} \ell_j^{\text{GT}}}^{-1}, \quad (29)$$

where the calibration matrices are given in (28).

Global Error The global error measures the difference between an estimated pose relative to the initial pose and the ground truth. It evaluates the overall estimation accuracy along a trajectory and is calculated at a given time using (29),

$$\text{GE}(\ell, t_k) := \ln(\text{err}_\ell(t_k, t_1))^\vee.$$

The maximum global errors for each estimator are given in Tables 2, 3 and 7 and illustrated in Figs. 8–15.

The global error on a specific data segment is contextualized by the total ground-truth path length (Tables 4–6) and maximum trajectory displacement (Tables 2, 3 and 7). The total path contextualizes how far a motion travels in all directions, but does not completely describe the trajectory or the difficulty in estimating it. A repetitive motion, such as swinging blocks, unwinds accumulated error and makes the total path length less relevant for evaluating the estimator. The maximum trajectory displacement contextualizes how far a motion travels in any given direction. Unlike the total path length, this metric does not differentiate between a single traversal of a given path and multiple traversals. Together, both values are useful in understanding the global error of a given estimator on a particular data segment.

Relative Error The global error is sensitive to the time at which an error occurs. An estimation error that occurs at the beginning of a trajectory results in significantly more global error than an equivalent error occurring at the end of the trajectory. In contrast, the relative error measures how much the estimated transform between consecutive time stamps deviates from the ground truth. The translational and rotational root-mean-square (RMS) relative errors are given by

$$\begin{aligned} \text{RMSRE}_{\text{xyz}}(\ell) &:= \sqrt{\frac{1}{K} \sum_{k=2}^K \|\text{xyz}(\rho_{\ell,k})\|}, \\ \text{RMSRE}_{\text{angle}}(\ell) &:= \sqrt{\frac{1}{K} \sum_{k=2}^K \|\phi_{\ell,k}\|}, \end{aligned}$$

where $\text{xyz}(\cdot)$ converts the $\mathfrak{se}(3)$ translational element to the equivalent Euclidean translation, and

$$\xi_{\ell,k} := \begin{bmatrix} \rho_{\ell,k} \\ \phi_{\ell,k} \end{bmatrix} := \ln(\text{err}_\ell(t_k, t_{k-1}))^\vee.$$

This provides a single measure of the average relative error independent of when an individual error occurs.

The RMSRE is compared to the RMS ground-truth frame-to-frame motion in Tables 8–10. The ground-truth relative motion illustrates how much each object moves between each time stamp, which can become arbitrarily small for high-frame-rate sensors, even for quickly moving objects. Small ground-truth relative motions can result in estimators having arbitrarily small relative error.

Both global and relative errors are decomposed into their translational (i.e., xyz) and rotational (i.e., roll-pitch-yaw) components. All errors are reported for geocentric trajectory estimates, so a portion of the error for each motion is due to error in the camera motion estimate.

VDO-SLAM third-party motion estimates were provided as calibrated relative motions. These transforms are recalibrated and combined to match the global transform representation of MVO for evaluation.

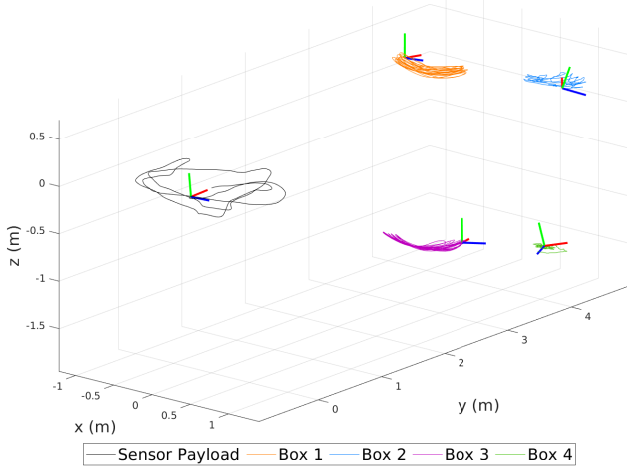


Figure 5. Motion segmentation (top) and trajectories (bottom) produced by MVO using the pose-velocity estimator for the `swinging_4_unconstrained` data segment from the OMD. The egomotion (black, bottom) of the camera is estimated from the static points in the scene (black, top). The motions of the swinging blocks (1–4) are segmented and estimated simultaneously with the egomotion. The results for each MVO estimator for this segment are illustrated in Extensions 1–3 (Appendix A).

6.3 Multimotion Estimation

The `swinging_4_unconstrained` segment of the OMD presents complex $SE(3)$ motions without significant occlusion. It tests the ability of estimators to simultaneously segment and estimate multiple motions. It consists of four blocks that both translate and rotate in pendular motions (Fig. 5). The blocks are observed by a dynamic camera, resulting in five independent $SE(3)$ motions. A 500-frame section starting at frame 10 was used in this evaluation.

Two of the blocks, Block 1 (top left) and Block 3 (bottom left), have minimal rotation and swing primarily along the ground-truth world y - and x -axis, respectively. The other two blocks, Block 2 (top right) and Block 4 (bottom right), rotate significantly about the positive ground-truth body z -axis. Block 4 rotates counter-clockwise and Block 2 rotates clockwise while also swinging, which creates a complex motion.

6.4 Tracking Through Occlusion

The `occlusion_2_unconstrained` segment of the OMD presents multiple $SE(3)$ motions with significant occlusions (Fig. 6). It tests the ability of estimators to segment

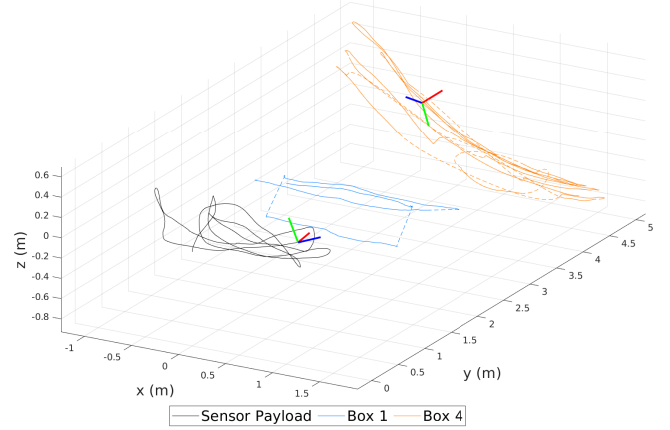
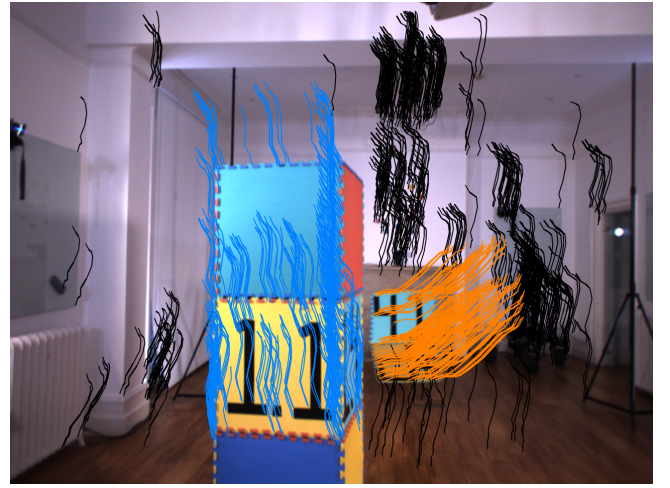


Figure 6. Motion segmentation (top) and trajectories (bottom) produced by MVO using the pose-velocity estimator for the `occlusion_2_unconstrained` data segment from the OMD. The egomotion (black, bottom) of the camera is estimated from the static points in the scene (black, top). The motions of the swinging block (4, orange) and the block tower (1, blue) are segmented and estimated simultaneously with the egomotion. The results for each MVO estimator for this segment are illustrated in Extensions 4–6 (Appendix A).

and estimate repeatedly occluded motions. It includes three independent motions, a sliding block tower (Block 1) that occasionally becomes static, a swinging block (Block 4) that is repeatedly occluded by the tower, and the camera egomotion. A 500-frame section starting at frame 1750 was used in this evaluation. The segment is particularly challenging because it includes multiple instances of the swinging block changing direction when partially or fully occluded.

The sliding block tower is partially occluded multiple times when it partially leaves the field of view of the camera, further complicating the estimation. It also occasionally stops moving and becomes part of the static background, which is a form of indirect occlusion in motion-based tracking. VDO-SLAM was not evaluated on this segment because it is not designed for this level of occlusion.

6.5 Urban Driving

The KITTI Vision Benchmark Suite (Geiger et al. 2012) is a collection of datasets and benchmarks dedicated to autonomous driving scenarios. The suite contains data and evaluation metrics for object detection and tracking, depth and flow calculations, and VO. The datasets and benchmarks

Table 2. Maximum translational and roll-pitch-yaw errors for each MVO estimator and VDO-SLAM for the `swinging_4_unconstrained` data segment from the OMD. The maximum ground-truth trajectory displacement is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$) and are not bounded to $\pm 180^{\circ}$. Bolded values indicate the best results for each estimated quantity.

Obj.	Pose-Only				Pose-Velocity				Pose-Velocity-Acceleration				VDO-SLAM				Max Ground Truth Displacement			
	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw
Ego	0.08	-1.46	-0.63	1.16	0.08	-1.48	-0.63	1.19	0.08	-1.40	-0.72	1.15	0.46	-2.74	2.07	2.74	1.07	-5.76	-23.44	-14.58
B. 1	0.09	10.39	-3.99	-1.31	0.10	9.29	-3.01	-1.43	0.57	2.83	7.24	-3.96	1.06	-33.13	38.80	24.90	0.87	27.77	-33.96	-18.96
B. 2	0.31	11.12	-5.46	-67.07	0.22	15.39	-7.47	-80.16	0.19	11.63	-4.68	-62.68	0.40	12.29	13.62	-191.42	0.26	-9.38	11.16	-1115.85
B. 3	0.13	2.59	-3.02	3.65	0.12	2.73	-2.65	2.50	0.14	8.46	5.51	-8.57	1.30	10.16	-14.57	7.08	0.71	-30.88	-35.15	36.19
B. 4	0.55	14.80	7.06	91.69	0.41	9.52	-9.57	112.42	0.19	8.67	3.65	83.54	0.76	-38.78	27.03	211.81	0.22	-4.10	6.34	1484.20

Table 3. Maximum translational and roll-pitch-yaw errors for each MVO estimator for the `occlusion_2_unconstrained` data segment from the OMD, as well as the maximum extrapolated errors for each block when occluded. The maximum ground-truth trajectory displacement is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$) and are not bounded to $\pm 180^{\circ}$. Bolded values indicate the best results for each estimated quantity.

Object	Pose-Only				Pose-Velocity				Pose-Velocity-Acceleration				Max Ground Truth Displacement			
	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw
Ego	0.32	-4.64	1.79	-4.31	0.12	-5.02	-2.12	3.10	0.21	-4.95	-1.64	-3.43	1.32	-8.40	24.81	-13.48
Block 1	0.53	-3.69	-20.32	33.56	0.46	1.89	3.89	-8.57	0.56	-10.10	-9.68	12.13	1.68	-3.04	1.00	16.31
extrapolated	1.32	-3.69	-20.32	33.56	0.46	1.89	4.61	-8.57	0.64	-10.10	-9.68	12.697				
Block 4	0.59	129.16	74.63	65.37	0.58	135.70	58.90	61.27	0.92	-184.36	-83.17	161.91	2.22	-54.24	-38.62	32.82
extrapolated	1.13	129.16	74.63	65.37	1.39	135.70	58.90	61.96	3.88	-184.36	-83.17	161.91				

Table 4. Total ground-truth translational and roll-pitch-yaw path length for the `swinging_4_unconstrained` data segment from the OMD. Translational xyz motions is given in meters (m) and rotational roll-pitch-yaw motion is given in degrees ($^{\circ}$).

Object	Total Path			
	xyz	roll	pitch	yaw
Ego	13.71	181.92	262.91	180.29
Block 1	26.65	662.60	967.34	195.32
Block 2	8.12	334.43	141.51	1115.85
Block 3	18.91	280.44	1116.73	135.41
Block 4	2.79	74.30	63.84	1484.20

Table 5. Total ground-truth translational and roll-pitch-yaw path length for the `occlusion_2_unconstrained` data segment from the OMD. Translational xyz motions is given in meters (m) and rotational roll-pitch-yaw motion is given in degrees ($^{\circ}$).

Object	Total Path			
	xyz	roll	pitch	yaw
Ego	12.26	319.06	314.98	241.32
Block 1	6.72	55.68	27.78	268.58
Block 4	21.72	530.96	496.46	113.44

Table 6. Total ground-truth translational and roll-pitch-yaw path length for the `Drive 0005` and `Odometry 03` data segments from the KITTI Vision Benchmark Suite. Translational xyz motions is given in meters (m) and rotational roll-pitch-yaw motion is given in degrees ($^{\circ}$).

Object	Total Path			
	xyz	roll	pitch	yaw
Drive 0005	69.43	16.47	15.59	137.59
Odometry 03	561.09	98.69	90.45	275.66

Table 7. Maximum translational and roll-pitch-yaw errors for each MVO estimator and VDO-SLAM for the egomotion in the `Drive 0005` and `Odometry 03` data segments from the KITTI Vision Benchmark Suite. The maximum ground-truth trajectory displacement is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$) and are not bounded to $\pm 180^{\circ}$. Bolded values indicate the best results for each estimated quantity. VDO-SLAM results for the `Odometry 03` segment were not available.

Segment	Pose-Only				Pose-Velocity				Pose-Velocity-Acceleration				VDO-SLAM				Max Ground Truth Displacement			
	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw	xyz	roll	pitch	yaw
Drive 0005	3.17	-0.71	-0.29	-0.59	3.26	-1.07	0.54	-0.65	3.22	-0.77	-0.33	-1.03	4.17	1.94	-2.03	2.42	62.25	-2.42	1.56	48.40
Odometry 03	29.0	-2.19	-1.25	-5.60	31.4	-2.34	1.00	-6.27	30.1	-2.14	1.67	-5.72					511.87	4.09	-7.78	-92.04

Table 8. RMS (and maximum) relative translational and axis-angle errors for each MVO estimator and VDO-SLAM for the `swinging_4_unconstrained` data segment from the OMD. The RMS ground-truth frame-to-frame motion is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$). Bolded values indicate the best results for each estimated quantity.

Object	Pose-Only		Pose-Velocity		Pose-Velocity-Acceleration		VDO-SLAM		RMS Ground Truth	
	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle
Ego	0.007 (0.018)	0.293 (1.010)	0.002 (0.007)	0.240 (1.590)	0.002 (0.007)	0.263 (1.752)	0.010 (0.032)	0.579 (2.568)	0.029 (0.046)	0.931 (2.512)
Block 1	0.012 (0.045)	1.218 (5.579)	0.005 (0.014)	0.228 (0.924)	0.005 (0.018)	0.322 (1.394)	0.023 (0.113)	1.191 (4.334)	0.058 (0.103)	2.587 (5.210)
Block 2	0.007 (0.024)	0.350 (1.683)	0.007 (0.032)	0.632 (2.819)	0.005 (0.019)	0.453 (1.975)	0.012 (0.041)	0.733 (4.161)	0.018 (0.030)	2.423 (3.223)
Block 3	0.013 (0.055)	1.361 (6.057)	0.005 (0.014)	0.234 (1.220)	0.005 (0.012)	0.343 (1.243)	0.012 (0.040)	0.682 (1.630)	0.042 (0.077)	2.637 (5.520)
Block 4	0.003 (0.009)	0.241 (1.583)	0.007 (0.025)	0.747 (3.304)	0.005 (0.018)	0.537 (2.476)	0.019 (0.088)	0.884 (3.308)	0.006 (0.011)	2.989 (3.608)

Table 9. RMS (and maximum) relative translational and axis-angle errors for each MVO estimator for the `occlusion_2_unconstrained` data segment from the OMD. The RMS ground-truth frame-to-frame motion is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$). Bolded values indicate the best results for each estimated quantity.

Object	Pose-Only		Pose-Velocity		Pose-Velocity-Acceleration		RMS Ground Truth	
	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle
Ego	0.009 (0.125)	0.997 (6.652)	0.005 (0.033)	0.995 (6.642)	0.006 (0.050)	1.006 (6.714)	0.026 (0.015)	0.697 (2.036)
Block 1	0.019 (0.186)	0.887 (9.871)	0.014 (0.101)	0.343 (2.659)	0.019 (0.147)	0.673 (7.195)	0.015 (0.026)	1.829 (5.332)
extrapolated	0.026 (0.780)	0.993 (7.156)	0.014 (0.101)	0.379 (2.658)	0.019 (0.147)	0.659 (6.610)		
Block 4	0.026 (0.158)	1.123 (7.537)	0.027 (0.124)	1.198 (4.732)	0.026 (0.133)	2.321 (12.97)	0.050 (0.119)	1.436 (5.773)
extrapolated	0.029 (1.123)	1.303 (4.411)	0.029 (0.133)	1.346 (6.100)	0.044 (0.471)	2.643 (7.569)		

Table 10. RMS (and maximum) relative translational and axis-angle errors for each MVO estimator and VDO-SLAM for the `Drive 0005` and `Odometry 03` data segments from the KITTI Vision Benchmark Suite. The RMS ground-truth frame-to-frame motion is also included for comparison. Translational xyz errors are given in meters (m) and rotational roll-pitch-yaw errors are given in degrees ($^{\circ}$). VDO-SLAM results for the `Odometry 03` segment were not available.

Segment	Pose-Only		Pose-Velocity		Pose-Velocity-Acceleration		VDO-SLAM		RMS Ground Truth	
	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle	xyz	axis-angle
Drive 0005	0.050 (0.093)	0.083 (0.180)	0.052 (0.101)	0.077 (0.166)	0.051 (0.130)	0.081 (0.279)	0.063 (0.160)	0.087 (0.189)	0.467 (0.713)	0.992 (1.587)
Odometry 03	0.156 (0.973)	0.080 (0.294)	0.145 (0.723)	0.090 (0.825)	0.164 (1.253)	0.091 (0.959)			0.738 (0.972)	0.642 (3.260)

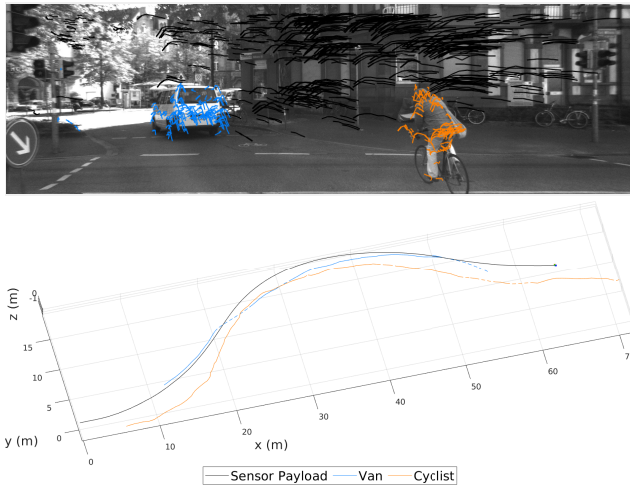


Figure 7. Motion segmentation (top) and trajectories (bottom) produced by MVO using the pose-velocity estimator for the Drive 0005 data segment from the KITTI Vision Benchmark Suite. The egomotion (black, bottom) of the camera is estimated from the static points in the scene (black, top). The motions of the cyclist (orange) and the van (blue) are segmented and estimated simultaneously with the egomotion. These results are illustrated in Extension 7 (Appendix A).

within KITTI are widely used to develop and evaluate computer vision techniques, but it does not contain any ground-truth third-party trajectory data. This makes it less suitable for quantitatively evaluating multimotion estimation algorithms; however, it is still a useful qualitative tool for evaluation in a real-world scenario.

The KITTI Drive 0005 segment (Fig. 7) is a 154-frame sequence that contains two independent motions, a cyclist and van, observed by a car-mounted camera as they travel around two consecutive turns. The van is temporarily occluded multiple times as it rounds a turn, is obscured by the cyclist, or is overexposed by sunlight. These all contribute to a challenging tracking and estimation problem that is compounded by the significant distance between the van and the camera.

The KITTI Odometry 03 segment, is a longer 800-frame sequence that only contains one independent motion, another car, observed for part of the segment. This segment is useful for evaluating egomotion estimation over a significant distance but is not particularly interesting for multimotion estimation.

7 Discussion

Qualitative and quantitative evaluations show that MVO consistently estimates ego- and third-party motions, even in the presence of temporary occlusions. MVO demonstrates good estimation and tracking accuracy on the evaluated toy problems and a real-world driving scenario without relying on appearance-based detectors or preprocessing segmentation stages.

MVO uses sparse feature tracking which does not require prior knowledge about object appearance but has known limitations (Section 7.1). The distribution of features on each object affects the estimation of its motion, and small or distant objects can be difficult to segment and estimate. Similarly, MVO demonstrates that motion-based tracking through occlusions is a reliable way to address the MEP without making

appearance-based assumptions, but accurately estimating the rotation of a previously occluded object without modeling its shape is challenging. Some of the limitations of MVO are inherent to the estimators it employs, each bringing its own advantages and disadvantages (Section 7.2). Despite its limitations, MVO outperforms VDO-SLAM in both egomotion and third-party trajectory estimation accuracy (Section 7.3).

7.1 Limitations of Sparse Approaches

MVO is a sparse, feature-based approach and therefore its estimation accuracy depends on the distribution of features on each dynamic object. This distribution affects the nature of the estimated motion, the observed shape of the object, and the motion extrapolation during occlusions.

Features are only observed on object surfaces facing the sensor, which makes it difficult to estimate certain types of motions, such as translations along the optical axis and rotations not around the optical axis. For example, Block 2 and Block 4 in the `swinging_4.unconstrained` segment (Figs. 8–10) both rotate significantly around the z -axis, leading to larger rotational errors. Not only are these rotations outside the image plane, but it is also difficult to differentiate between an object rotating in place and orbiting about an offset axis. These challenges could be overcome by using longer estimation windows or incorporating 3D fusion techniques (Rünz and Agapito 2017). Despite this, the yaw errors for both Block 2 and Block 4 are less than 7.5% of their total yaw rotation for all MVO estimators.

The observable shape (and centroid) of an object also changes over time as it and the sensor move. This shifts the observable features on the object and the geocentric trajectory estimate, which degrades the interpolated estimates. This is often most evident before and after an occlusion, as seen in the estimates for the block tower in the `occlusion_2.unconstrained` segment (Figs. 11b, 12b and 13b). The block tower often extends outside the camera frustum while stopped (i.e., occluded for motion-based tracking), which leads to a large discrete jump in its observed centroid when it is tracked again. These discrete jumps create interpolation errors during motion closure and lead to considerably higher global errors than would otherwise be expected given the small relative error (Tables 3 and 9)

The transition into occlusion itself presents estimation challenges. As the object becomes more occluded, the number of tracked features decreases and the quality of the segmentation and estimation degrades. This is evident in the egomotion and block tower estimates for the `occlusion_2.unconstrained` segment, which are occasionally corrupted when the block tower slows to a stop and an increasing number of its features are incorrectly classified as part of the background (Figs. 11–13).

The transition into occlusion also makes extrapolation harder, because the erroneous estimates before an occlusion are extrapolated forward and make motion closure more difficult. This is evident in the swinging block estimates for the `occlusion_2.unconstrained` segment, which are often poorly extrapolated (Figs. 11c, 12c and 13c). Objects rarely become occluded or unoccluded instantaneously and performance in the transitional periods could be improved by predicting occlusions (Mitzel et al. 2010) or more sophisticated centroid calculation (Setterfield et al. 2018).

7.2 Estimator Comparison

MVO is a direct extension of VO to the MEP and can therefore leverage significant developments in egomotion estimation. This includes different $SE(3)$ estimation techniques, such as the discrete, WNOA, and WNOJ motion models presented here. The associated estimators have advantages and disadvantages in MVO.

The pose-only estimator is the most general of those discussed in this paper. Its lack of motion prior makes it simple, but it can estimate physically implausible motions (e.g., discrete jumps) and it is not as robust to occlusions. The pose-velocity and pose-velocity-acceleration estimators constrain the trajectory estimate with a WNOA and WNOJ prior, respectively. This makes estimation more complex but penalizes physically implausible motions and allows for more accurate extrapolation and interpolation through occlusions. The WNOA prior assumes a locally constant velocity that is appropriate for many real-world situations but does not accurately model motions with large changes in direction. The WNOJ prior assumes a locally constant acceleration that is applicable to more real-world situations, including objects with large changes in direction; however, it often does not sufficiently constrain its estimates to accurately extrapolate motions for long periods of time.

7.2.1 Swinging Blocks Segment All estimators performed similarly well in the `swinging_4_unconstrained` segment, which is the simplest scenario without occlusion (Section 6.3). They all consistently segmented each motion and accurately estimated the motion of the camera and Blocks 1 and 3 but underestimated the yaw rotation of Blocks 2 and 4 (Figs. 8–10 and Tables 2, 4 and 8). This yaw rotation is an out-of-plane rotation that is hard to observe and its estimation may be improved with appearance-based object models. The global and relative estimation errors for the different estimators and VDO-SLAM are given in Tables 2 and 8 and shows that the MVO estimators all perform similarly while outperforming VDO-SLAM.

7.2.2 Occlusion Segment The estimators perform differently in the presence of occlusions, as seen in the `occlusion_2_unconstrained` results (Section 6.4). MVO consistently estimates the motions of both the camera and the moving blocks through motion closure with all estimators; however, the newly calculated centroid of a motion after an occlusion does not always match that of the original motion. This discrepancy causes jumps in the trajectory, as the original centroid is projected forward to a different location on the object than previously estimated. This affects the calculation of the correction transform (Section 5.3.2) and is a major source of error in the estimates of the block tower, as it is often partially outside the view of the camera. The global and relative estimation errors for the different estimators are given in Tables 3 and 9 and show that the pose-velocity estimator was generally the most accurate.

The pose-only estimator performed reasonably well for directly observed motions but did not perform well on occlusions (Fig. 11 and Tables 3, 5 and 9). This is because its linear extrapolation and interpolation are sensitive to estimation noise, which often increases as an object becomes occluded (Section 7.1). The egomotion estimate is also affected by the sliding block transitioning between being

stationary and dynamic more than the other estimators because of the lack of motion prior.

The pose-velocity estimator performed well for the entire segment and was the best at extrapolating occluded motions (Fig. 12 and Tables 3, 5 and 9). Its locally constant-velocity assumption reasonably models many motions and allows for gradual changes in direction when directly observed. When changes in motion are not observed (i.e., during occlusion), the extrapolation can be inaccurate and motion closure may fail. This is expected because a WNOA prior only allows for small changes in velocity between consecutive frames, so it distributes large changes in motion over longer periods of time.

The pose-velocity-acceleration estimator was the best at interpolating occluded motions, but occasionally underconstrained its extrapolated estimates (Fig. 13 and Tables 3, 5 and 9). Its locally constant-acceleration assumption directly models changes in velocity, even during occlusion. This more expressive prior results in better interpolations after motion closure than the WNOA prior but is less robust when extrapolating motions. This is because even small nonzero accelerations are extrapolated to significant motions over a moderate-length occlusion, as evidenced by the high extrapolation error for the swinging block.

All estimators had high rotational error after occlusion because motion closure does not correct the extrapolated rotation (Section 5.3). This results in similar maximum rotation errors when extrapolating and interpolating (Tables 2, 3 and 7). Accurately recognizing the relative rotation of an object after an occlusion is difficult without modeling the appearance or structure of the object. This may be achieved by tracking the visual features of the object or by building a model of its structure.

7.2.3 KITTI Segments MVO consistently estimates the motions of the camera and the cyclist throughout the `Drive 0005` segment, and the estimators perform similarly for egomotion estimation (Section 6.5). The van is directly or indirectly occluded several times, but MVO reliably extrapolates and tracks its motion through motion closure for most of the segment until the van becomes too far away for accurate stereo triangulation. VDO-SLAM demonstrates a similar ability to track the cyclist and the van.

The egomotion estimation error for the different estimators and VDO-SLAM on the `Drive 0005` and `Odometry 03` segments is given in Tables 7 and 10 and is less than 5.6% of path length for all MVO estimators. These errors represent a similar percentage of the ground-truth motion as in the toy problems (Tables 6, 7 and 10 and Figs. 14 and 15). The segment does not include ground-truth trajectory data for third-party motions, so it is not useful for comparing the estimators beyond egomotion accuracy.

The decision of which estimator to use in any given application can be dependent on the requirements and characteristics of the situation. Based on the performance demonstrated on these segments, the WNOA is likely the most reliable prior for the general MEP. The WNOJ prior proved most accurate for interpolation, and it could likely be combined with the WNOA estimator for improved performance in cases with significant occlusion of nonconstant motions.

7.3 MVO Comparison

MVO and VDO-SLAM demonstrate similar tracking performance, but MVO outperforms VDO-SLAM for metric estimation of both ego- and third-party motions for the `swinging_4_unconstrained` (Section 6.3) and `Drive 0005` (Section 6.5). MVO achieves this without relying on learning-based preprocessing steps or other appearance-based information and can track motions through significant occlusions.

VDO-SLAM prioritizes estimating relative object motions in outdoor scenes with minimal rotation, such as autonomous driving scenarios, and is not designed for significant occlusion, so it is not evaluated on `occlusion_2_unconstrained`. It is most comparable to MVO in egomotion accuracy, especially for the `Drive 0005` segment (Tables 7 and 10), but this design focus limits its applicability to the general MEP.

Unlike VDO-SLAM, MVO is reliant on motion, so objects that temporarily move similarly may be given the same label, such as when the sliding block tower becomes stationary (Section 6.4). Extrapolating lost motions even when they are similar to existing motions allows temporarily similar motions to be resegmented once they diverge, providing a form of *motion permanence*. This allows the block tower to be successfully tracked through motion-based occlusions, but MVO still had higher egomotion errors in the `occlusion_2_unconstrained` segment than the `swinging_4_unconstrained` segment because the block tower’s transitions between dynamic and stationary motion created noise in the egomotion label. VDO-SLAM uses front-end instance segmentation to avoid the difficulty of distinguishing between similarly moving objects, instead identifying objects to track based on appearance. Incorporating similar appearance-based techniques into MVO could improve its performance when independent objects temporarily move similarly.

MVO and VDO-SLAM qualitatively demonstrate a similar ability to track both the cyclist and van in the `Drive 0005` segment. Both consistently segment and estimate the cyclist until failing to track the van at the same time in the segment, though metric comparisons of their third-party estimates are not possible for this dataset due to its lack of ground-truth data. VDO-SLAM achieves this by relying on separate preprocessing algorithms to perform instance segmentation and optical flow calculation. MVO has no such dependencies, segmenting the scene and estimating trajectories based on the bulk motion of tracked feature points. This means it is applicable to a variety of different environments without significant tuning or prior knowledge of object number, appearance, or motion.

8 Conclusion

This paper presents MVO as an extension of the traditional VO pipeline to address the MEP. It uses multilabeling techniques to simultaneously segment and estimate all rigid motions in a scene. A variety of motion-based trajectory states are explored, and their effect on the ability to estimate independent *SE* (3) motions and track them through occlusions is discussed.

A stereo implementation of MVO is evaluated both quantitatively and qualitatively using the OMD (Judd and

Gammell 2019a) and the KITTI Vision Benchmark Suite (Geiger et al. 2012). MVO achieves egomotion estimation accuracy comparable to similarly defined egomotion-only VO systems, and it outperforms an appearance-based multimotion system (Zhang et al. 2020b) in addressing the MEP.

The pipeline operates directly on tracked 3D points and is agnostic to the type of sensor that generates them, so a variety of sensors and corresponding estimators can be employed in MVO (Judd 2019). MVO also relies exclusively on motion-based estimation and tracking techniques. Future work will include extending MVO to other sensors, such as LiDAR and event cameras, parallelizing the batch estimation in order to run at real time, and introducing application-specific, appearance-based techniques to improve performance in specific environments.

9 Acknowledgments

We would like to thank Paul Amayo for providing his implementation of CORAL and the authors of VDO-SLAM for providing their results and discussing them. We would also like to thank the editorial board for considering this manuscript and our reviewers for their time and constructive criticism.

This research was funded by UK Research and Innovation and EPSRC through Robotics and Artificial Intelligence for Nuclear (RAIN) [EP/R026084/1], ACE-OPS: From Autonomy to Cognitive assistance in Emergency OperationS [EP/S030832/1], and the Autonomous Intelligent Machines and Systems (AIMS) Centre for Doctoral Training (CDT) [EP/S024050/1].

A Index to Multimedia Extensions

The multimedia extensions in Table 11 are available at <https://www.youtube.com/c/roboticesp>.

B Nonlinear Stereo Camera Model

A calibrated stereo camera pair can be described by its intrinsic calibration matrix,

$$\mathbf{K} = \begin{bmatrix} f_u & 0 & u_0 & 0 \\ 0 & f_v & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

and its horizontal baseline b . The horizontal and vertical focal lengths, f_u and f_v , affect the camera field and depth of view. The principal point coordinates, u_0 and v_0 , define the camera center in the image plane. The nonlinear stereo camera projection, $\mathbf{s}_{\text{stereo}}(\cdot)$, projects a 3D world point onto the image plane according to

$$\begin{bmatrix} u \\ v \\ d \\ 1 \end{bmatrix} = \mathbf{s}_{\text{stereo}} \left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} \right) := \begin{bmatrix} \frac{f_u x}{z} - u_0 \\ \frac{f_v y}{z} - v_0 \\ \frac{f_u b}{z} \\ 1 \end{bmatrix},$$

where u and v are the horizontal and vertical left-image coordinates and d is the horizontal disparity between the image coordinate in the left and right image.

Table 11. Index to Multimedia Extensions.

Extension	Media Type	Description
1	Video	Results of the pose-only estimator on the swinging_4_unconstrained segment of the OMD
2	Video	Results of the pose-velocity estimator on the swinging_4_unconstrained segment of the OMD
3	Video	Results of the pose-velocity-acceleration estimator on the swinging_4_unconstrained segment of the OMD
4	Video	Results of the pose-only estimator on the occlusion_2_unconstrained segment of the OMD
5	Video	Results of the pose-velocity estimator on the occlusion_2_unconstrained segment of the OMD
6	Video	Results of the pose-velocity-acceleration estimator on the occlusion_2_unconstrained segment of the OMD
7	Video	Results of the pose-velocity estimator on the Drive 0005 segment of KITTI

This projection function has the corresponding Jacobian,

$$\mathbf{S}_{jk} := \frac{\delta \mathbf{s}}{\delta \mathbf{z}} \Big|_{\mathbf{z}(\mathbf{x}_{\text{op},jk})} = \begin{bmatrix} \frac{f_u}{z_3} & 0 & -\frac{f_u z_1}{z_3^2} & 0 \\ 0 & \frac{f_v}{z_3} & -\frac{f_v z_2}{z_3^2} & 0 \\ 0 & 0 & -\frac{f_u b}{z_3^2} & -\frac{f_u b}{z_3} \end{bmatrix},$$

where $\mathbf{z}(\mathbf{x}_{\text{op},jk}) = [z_1 \ z_2 \ z_3 \ 1]^T$. This Jacobian is used in the bundle adjustment optimization in Section 4 applied to the stereo camera data in Section 6.

References

- Agarwal S, Mierle K and Others (—) Ceres Solver.
- Amayo P, Piniés P, Paz LM and Newman P (2018) Geometric multi-model fitting with a convex relaxation algorithm. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Anderson S and Barfoot TD (2015) Full STEAM ahead: Exactly sparse Gaussian process regression for batch continuous-time trajectory estimation on SE(3). In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 157–164. DOI:10.1109/IROS.2015.7353368.
- Barfoot TD (2017) *State Estimation for Robotics*. Cambridge University Press.
- Bescos B, Campos C, Tardós JD and Neira J (2021) DynaSLAM II: Tightly-coupled multi-object tracking and SLAM. *IEEE Robotics and Automation Letters (RA-L)* 6(3): 5191–5198.

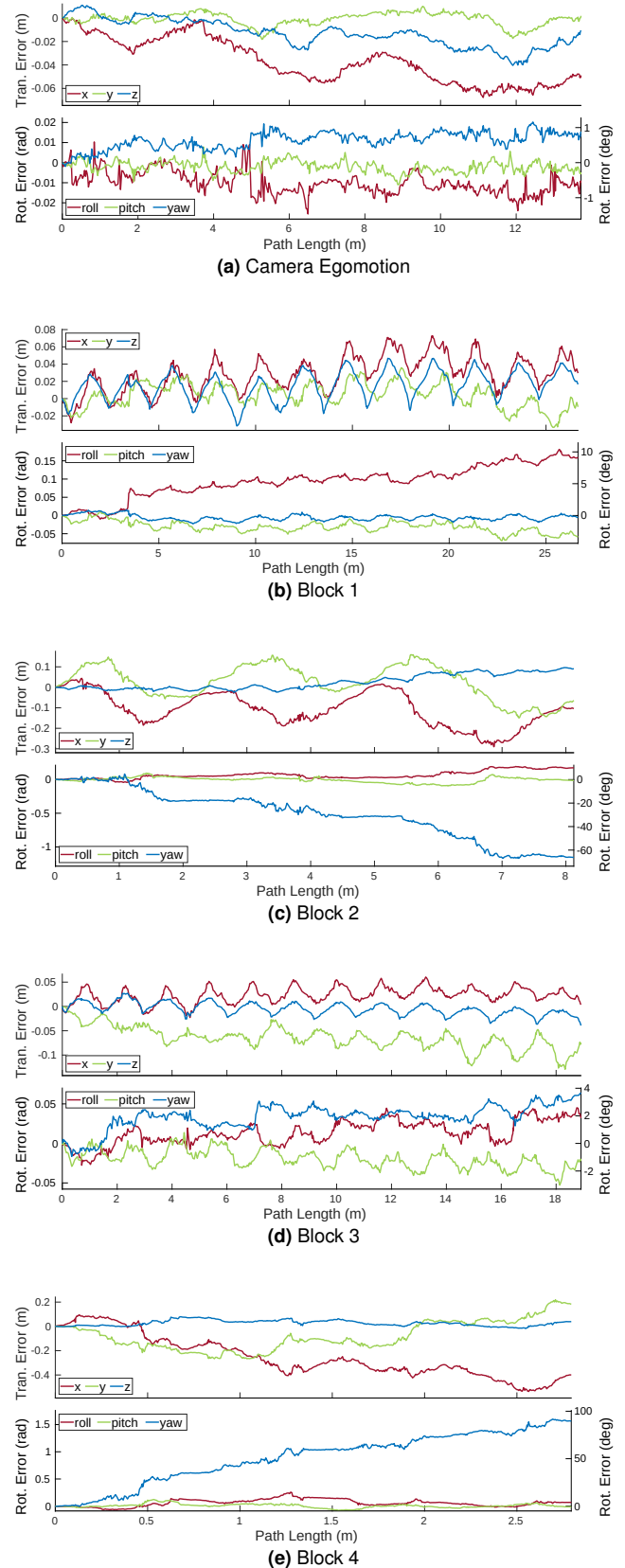
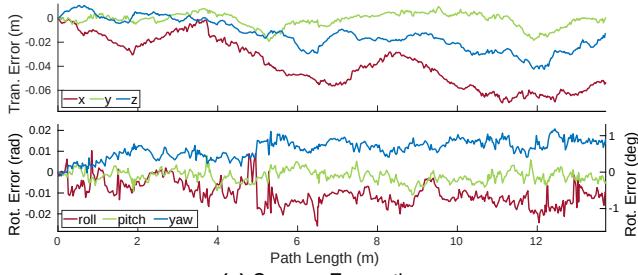
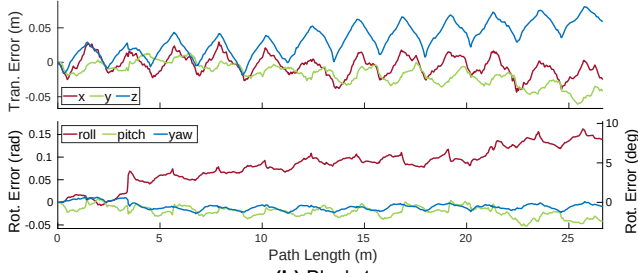


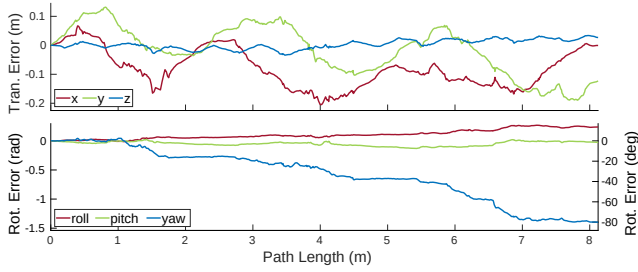
Figure 8. The translational and rotational errors estimated by MVO for the camera (a) and the swinging blocks (b–e) for a 500-frame section of the swinging_4_unconstrained segment from the OMD using the pose-only estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes.



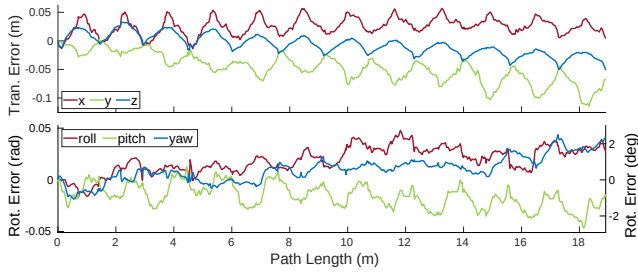
(a) Camera Egomotion



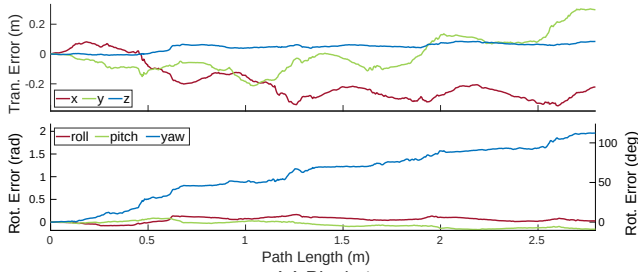
(b) Block 1



(c) Block 2

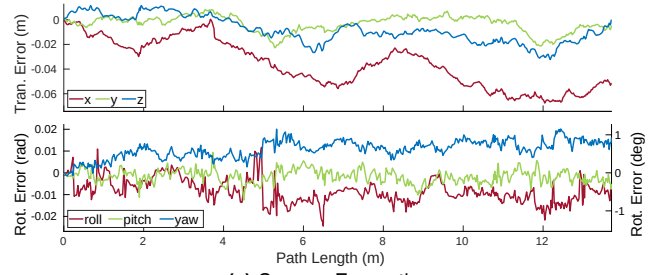


(d) Block 3

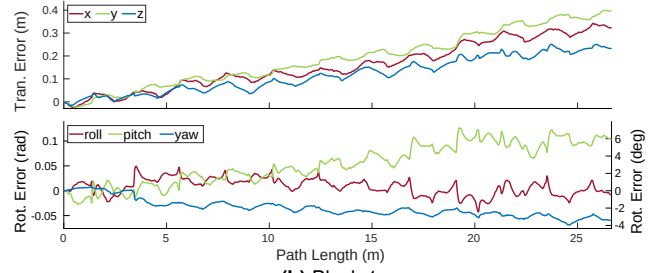


(e) Block 4

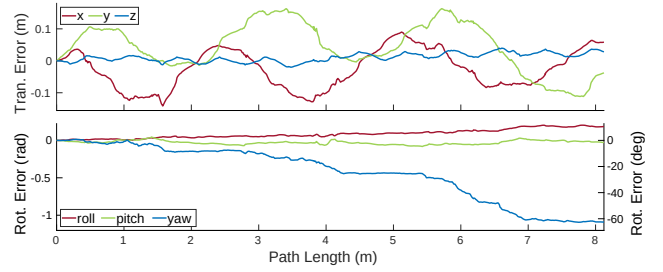
Figure 9. The translational and rotational errors estimated by MVO for the camera (a) and the swinging blocks (b–e) for a 500-frame section of the `swinging_4_unconstrained` segment from the OMD using the pose-velocity estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes.



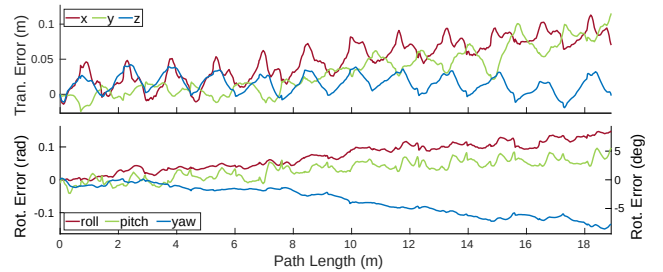
(a) Camera Egomotion



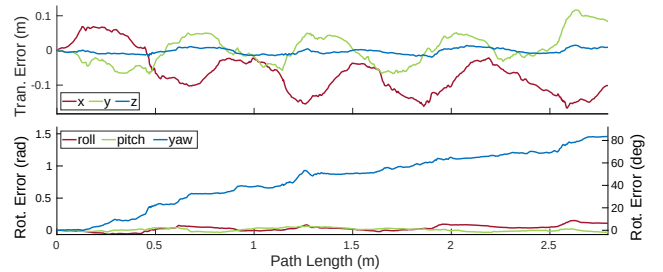
(b) Block 1



(c) Block 2

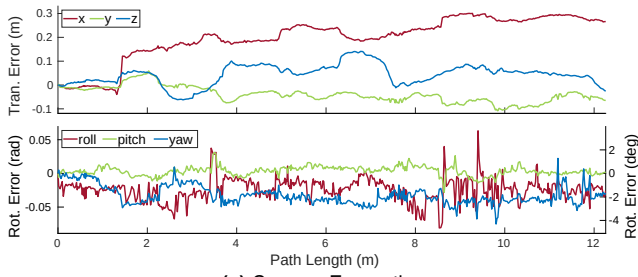


(d) Block 3

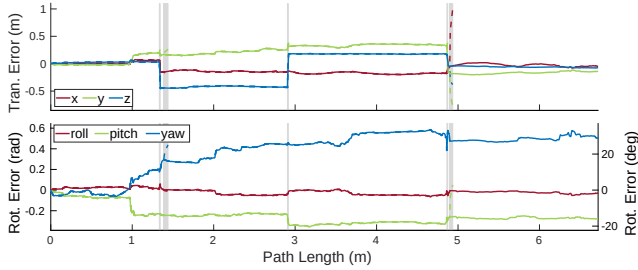


(e) Block 4

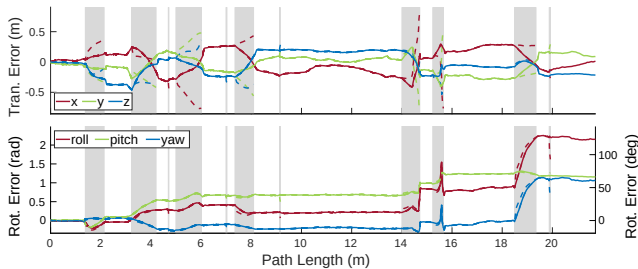
Figure 10. The translational and rotational errors estimated by MVO for the camera (a) and the swinging blocks (b–e) for a 500-frame section of the `swinging_4_unconstrained` segment from the OMD using the pose-velocity-acceleration estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes.



(a) Camera Egomotion

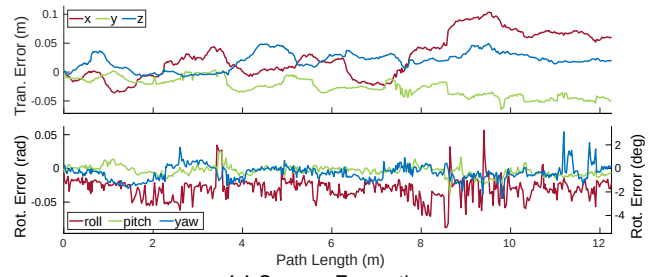


(b) Block Tower 1

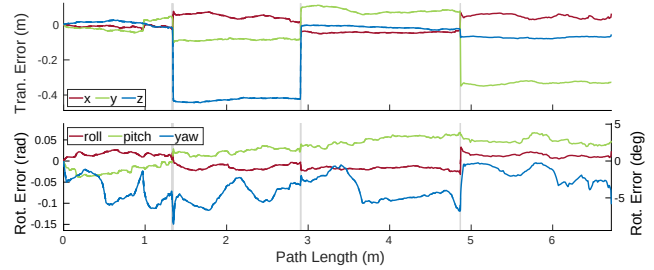


(c) Swinging Block 4

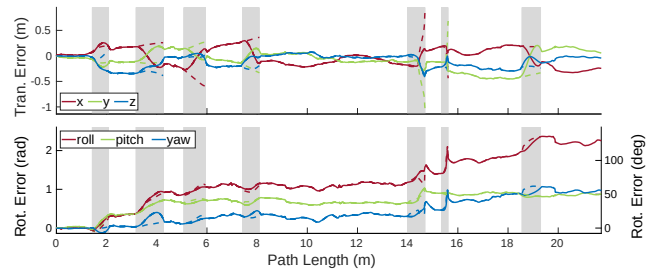
Figure 11. The translational and rotational errors estimated by MVO for the camera (a), the block tower (b), and the swinging block (c) for a 500-frame section of the `occlusion_2.unconstrained` segment from the OMD using the pose-only estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes. Grey regions show when the swinging block was occluded by the tower, or when the tower was stationary and effectively part of the background. Dashed lines represent the extrapolation error and the solid lines represent the error in the direct or interpolated estimates.



(a) Camera Egomotion



(b) Block Tower 1



(c) Swinging Block 4

Figure 12. The translational and rotational errors estimated by MVO for the camera (a), the block tower (b), and the swinging block (c) for a 500-frame section of the `occlusion_2.unconstrained` segment from the OMD using the pose-velocity estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes. Grey regions show when the swinging block was occluded by the tower, or when the tower was stationary and effectively part of the background. Dashed lines represent the extrapolation error and the solid lines represent the error in the direct or interpolated estimates.

Boykov Y, Veksler O and Zabih R (1999) Fast approximate energy minimization via graph cuts. In: *IEEE International Conference on Computer Vision (ICCV)*, volume 1. pp. 377–384.

Eckenhoff K, Yang Y, Geneva P and Huang G (2019) Tightly-coupled visual-inertial localization and 3-D rigid-body target tracking. *IEEE Robotics and Automation Letters (RA-L)* 4(2): 1541–1548. DOI:10.1109/LRA.2019.2896472.

Engel J, Koltun V and Cremers D (2017) Direct sparse odometry. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 40(3): 611–625.

Fischler MA and Bolles RC (1981) Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM* 24(6): 381–395.

Fraundorfer F and Scaramuzza D (2012) Visual odometry: Part II: Matching, robustness, optimization, and applications. *IEEE Robotics & Automation Magazine* 19(2): 78–90.

Geiger A, Lenz P and Urtasun R (2012) Are we ready for autonomous driving? The KITTI vision benchmark suite. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 3354–3361.

Geiger A, Ziegler J and Stiller C (2011) Stereoscan: Dense 3D reconstruction in real-time. In: *IEEE Intelligent Vehicles Symposium (IV)*. pp. 963–968.

Henein M, Zhang J, Mahony R and Ila V (2020) Dynamic slam: The need for speed. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 2123–2129.

Huang J, Yang S, Mu TJ and Hu SM (2020) ClusterVO: Clustering moving instances and estimating visual odometry for self and surroundings. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 2168–2177.

Huang J, Yang S, Zhao Z, Lai YK and Hu SM (2019) ClusterSLAM: A SLAM backend for simultaneous rigid body clustering and motion estimation. In: *Proceedings of the IEEE/CVF*

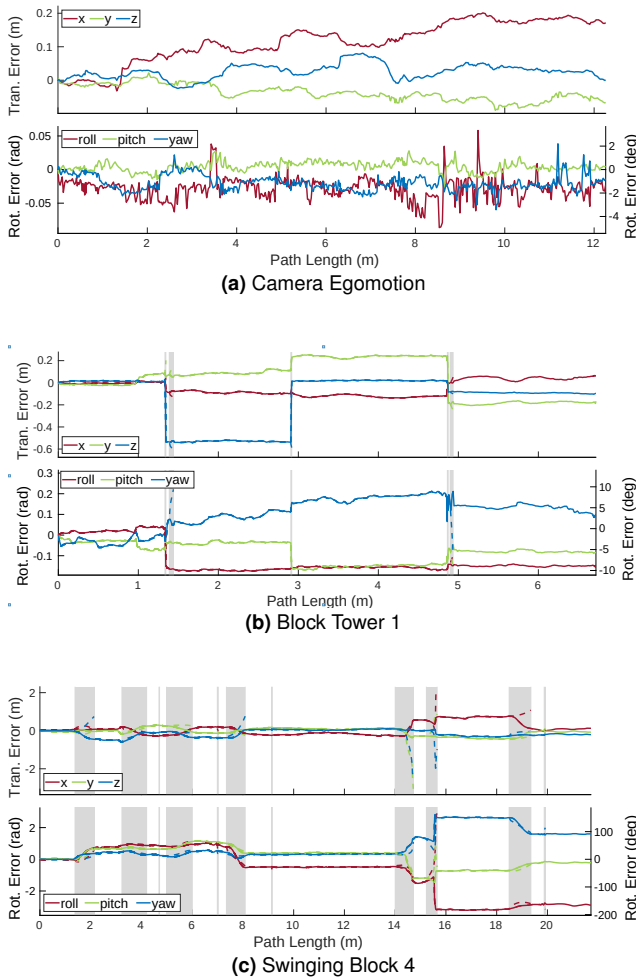


Figure 13. The translational and rotational errors estimated by MVO for the camera (a), the block tower (b), and the swinging block (c) for a 500-frame section of the `occlusion_2_unconstrained` segment from the OMD using the pose-velocity-acceleration estimator. Each object motion is compared to ground-truth trajectory data and errors are reported in an arbitrary geocentric frame with the z-axis up and arbitrary x- and y-axes. Grey regions show when the swinging block was occluded by the tower, or when the tower was stationary and effectively part of the background. Dashed lines represent the extrapolation error and the solid lines represent the error in the direct or interpolated estimates.

International Conference on Computer Vision. pp. 5875–5884.

Isack H and Boykov Y (2012) Energy-based geometric multi-model fitting. *International Journal of Computer Vision (IJCV)* 97(2): 123–147.

Jaimez M, Kerl C, Gonzalez-Jimenez J and Cremers D (2017) Fast odometry and scene flow from RGB-D cameras based on geometric clustering. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 3992–3999.

Judd K (2019) *Unifying motion segmentation, estimation, and tracking for complex dynamic scenes*. PhD Thesis.

Judd K and Gammell J (2019a) The oxford multimotion dataset: Multiple SE(3) motions with ground truth. *IEEE Robotics and Automation Letters (RA-L)* 4(2): 800–807. DOI:10.1109/LRA.2019.2892656. Presented at ICRA 2019.

Judd K and Gammell J (2019b) SE(3) multimotion estimation through occlusion. In: *Long-Term Human Motion Prediction (LHMP) Workshop, IEEE International Conference on Robotics*

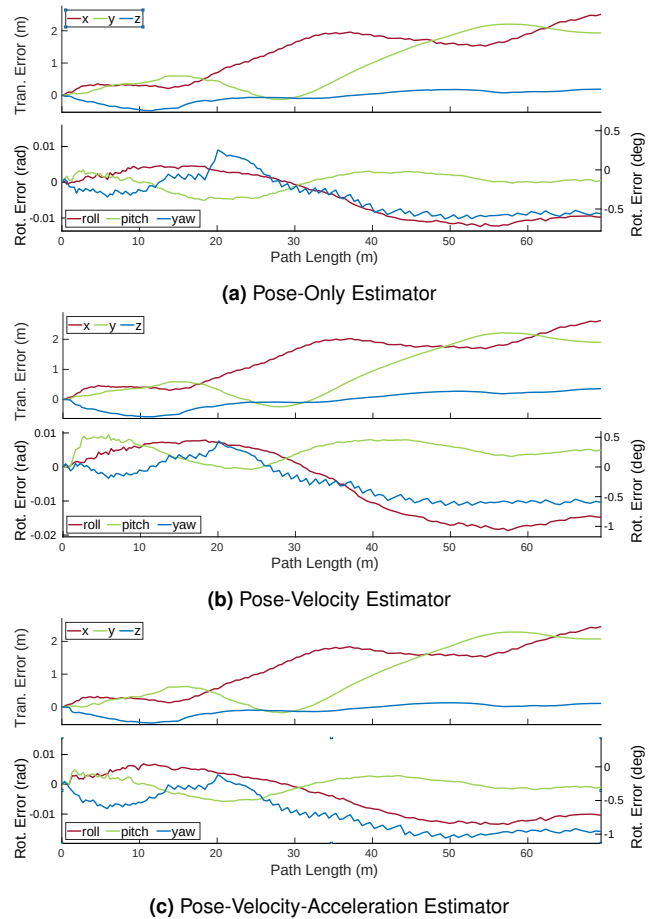


Figure 14. The translational and rotational errors estimated by MVO for the camera for the full 154-frame `Drive 0005` data segment from the KITTI Vision Benchmark Suite using each MVO estimator. There is no ground-truth trajectory data for the van or the cyclist.

and Automation (ICRA).

Judd K and Gammell J (2020) Occlusion-robust MVO: Multimotion estimation through occlusion via motion closure. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 5855–5862. DOI:10.1109/IROS45743.2020.9341355. ArXiv:1905.05121 [cs.RO].

Judd K, Gammell J and Newman P (2018a) Multimotion visual odometry (MVO): Simultaneous estimation of camera and third-party motions. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 3949–3956. DOI:10.1109/IROS.2018.8594213. ArXiv (corrected version):1808.00274 [cs.RO].

Judd K, Gammell J and Newman P (2018b) Visual multimotion estimation. In: *Joint Industry and Robotics CDTs Symposium (JIRCS)*.

Lenz P, Ziegler J, Geiger A and Roser M (2011) Sparse scene flow segmentation for moving object detection in urban environments. In: *IEEE Intelligent Vehicles Symposium (IV)*. pp. 926–932.

Lin KH and Wang CC (2010) Stereo-based simultaneous localization, mapping and moving object tracking. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 3975–3980.

Markley FL (1988) Attitude determination using vector observations and the singular value decomposition. *Journal of the Astronautical Sciences* 36(3): 245–258.

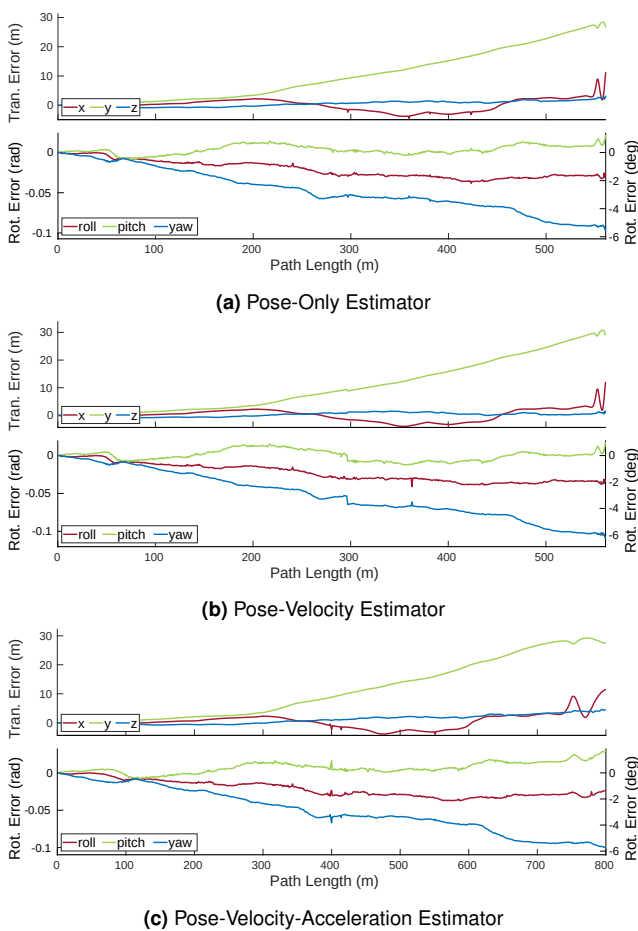


Figure 15. The translational and rotational errors estimated by MVO for the camera for the full 800-frame `Odometry_03` data segment from the KITTI Vision Benchmark Suite using each MVO estimator. There is no ground-truth trajectory data for the other car.

- Menze M and Geiger A (2015) Object scene flow for autonomous vehicles. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 3061–3070.
- Milan A, Leal-Taixé L, Reid ID, Roth S and Schindler K (2016) MOT16: A benchmark for multi-object tracking. *ArXiv: 1603.00831 [cs.CV]*.
- Mitzel D, Horbert E, Ess A and Leibe B (2010) Multi-person tracking with sparse detection and continuous segmentation. In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 397–410.
- Moravec HP (1980) *Obstacle Avoidance and Navigation in the Real World by a Seeing Robot Rover*. PhD Thesis, Stanford, CA, USA.
- Nistér D (2004) An efficient solution to the five-point relative pose problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 26(6): 756–777. DOI:10.1109/TPAMI.2004.17.
- Qiu K, Qin T, Gao W and Shen S (2019) Tracking 3-D motion of dynamic objects using monocular visual-inertial sensing. *IEEE Transactions on Robotics (T-RO)* 35(4): 799–816. DOI: 10.1109/TRO.2019.2909085.
- Quiroga J, Brox T, Devernay F and Crowley J (2014) Dense semi-rigid scene flow estimation from RGBD images. In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 567–582.
- Roussos A, Russell C, Garg R and Agapito L (2012) Dense multibody motion estimation and reconstruction from a handheld camera. In: *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. pp. 31–40.
- Rünz M and Agapito L (2017) Co-fusion: Real-time segmentation, tracking and fusion of multiple objects. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 4471–4478.
- Rünz M, Buffier M and Agapito L (2018) Maskfusion: Real-time recognition, tracking and reconstruction of multiple moving objects. In: *IEEE International Symposium on Mixed and Augmented Reality*. pp. 10–20. DOI:10.1109/InternationalSymposiumonMixedandAugmentedReality.2018.00024.
- Sabzevari R and Scaramuzza D (2016) Multi-body motion estimation from monocular vehicle-mounted cameras. *IEEE Transactions on Robotics (T-RO)* 32(3): 638–651.
- Setterfield TP, Miller DW, Leonard JJ and Saenz-Otero A (2018) Mapping and determining the center of mass of a rotating object using a moving observer. *The International Journal of Robotics Research* 37(1): 83–103.
- Tang TY, Yoon DJ and Barfoot TD (2019) A white-noise-on-jerk motion prior for continuous-time trajectory estimation on se (3). *IEEE Robotics and Automation Letters (RA-L)* 4(2): 594–601. DOI:10.1109/LRA.2019.2891492.
- Valgaerts L, Bruhn A, Mainberger M and Weickert J (2012) Dense versus sparse approaches for estimating the fundamental matrix. *International Journal of Computer Vision (IJCV)* 96(2): 212–234.
- Wahba G (1965) A least squares estimate of satellite attitude. *SIAM review* 7(3): 409–409.
- Wang C, Luo B, Zhang Y, Zhao Q, Yin L, Wang W, Su X, Wang Y and Li C (2020) DymSLAM: 4D dynamic scene reconstruction based on geometrical motion segmentation. *IEEE Robotics and Automation Letters (RA-L)* 6(2): 550–557.
- Wang CC, Thorpe C, Thrun S, Hebert M and Durrant-Whyte H (2007) Simultaneous localization, mapping and moving object tracking. *The International Journal of Robotics Research (IJRR)* 26(9): 889–916.
- Xu B, Li W, Tzoumanikas D, Bloesch M, Davison A and Leutenegger S (2019) Mid-fusion: Octree-based object-level multi-instance dynamic SLAM. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 5231–5237.
- Yang B, Huang C and Nevatia R (2011) Learning affinities and dependencies for multi-target tracking using a CRF model. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 1233–1240. DOI:10.1109/IEEE/CVFConferenceonComputerVisionandPatternRecognition.2011.5995587.
- Yang S and Scherer S (2019) CubeSLAM: Monocular 3D object SLAM. *IEEE Transactions on Robotics* 35(4): 925 – 938.
- Zhang J, Henein M, Mahony R and Ila V (2020a) Robust Ego and Object 6-DoF Motion Estimation and Tracking. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- Zhang J, Henein M, Mahony R and Ila V (2020b) VDO-SLAM: A visual dynamic object-aware SLAM system. *arXiv preprint arXiv:2005.11052*.